

В этом задании Вам предстоит обучить языковую модель с помощью рекуррентной нейронной сети. В отличие от семинарского занятия, Вам необходимо будет работать с отдельными словами, а не буквами.

Установим модуль `datasets`, чтобы нам проще было работать с данными.

```
# !pip install datasets
```

Импорт необходимых библиотек

```
import os
import random
import numpy as np
import torch
import sys

def set_seed(seed=42):
    """
    Фиксация источников случайности
    """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

    # Детерминированные алгоритмы
```

```
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    # Фиксация хэшей Python
    os.environ['PYTHONHASHSEED'] = str(seed)

    # Для CUDA
    os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':16:8'
    # os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'

# Фиксация случайности
SEED = 42
set_seed(SEED)

# import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# import numpy as np
import matplotlib.pyplot as plt

from tqdm.auto import tqdm
from datasets import load_dataset
from nltk.tokenize import sent_tokenize, word_tokenize
from sklearn.model_selection import train_test_split
import nltk

from collections import Counter
from typing import List

import math

import seaborn
seaborn.set_theme(palette='summer')

# nltk.download('punkt')

device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

```
'cuda'
```

```
NUM_EPOCH = 12
```

Подготовка данных

Воспользуемся датасетом imdb. В нем хранятся отзывы о фильмах с сайта imdb. Загрузим данные с помощью функции `load_dataset`

```
# Загрузим датасет
dataset = load_dataset('imdb')
```

Препроцессинг данных и создание словаря (1 балл)

Далее вам необходмо самостоятельно произвести препроцессинг данных и получить словарь или же просто `set` строк. Что необходимо сделать:

1. Разделить отдельные тренировочные примеры на отдельные предложения с помощью функции `sent_tokenize` из бибилиотеки `nltk`. Каждое отдельное предложение будет одним тренировочным примером.
2. Оставить только те предложения, в которых меньше `word_threshold` слов.
3. Посчитать частоту вхождения каждого слова в оставшихся предложениях. Для деления предлоения на отдельные слова удобно использовать функцию `word_tokenize`.
4. Создать объект `vocab` класса `set`, положить в него служебные токены '<unk>', '<bos>', '<eos>', '<pad>' и vocab_size самых частовстречающихся слов.

Посмотрим на примеры текстов

```
display(dataset['train']['text'][:7])
```

```
['I rented I AM CURIOUS-YELLOW from my video store because of all the
controversy that surrounded it when it was first released in 1967. I
also heard that at first it was seized by U.S. customs if it ever
tried to enter this country, therefore being a fan of films considered
"controversial" I really had to see this for myself.<br /><br />The
plot is centered around a young Swedish drama student named Lena who
wants to learn everything she can about life. In particular she wants
to focus her attentions to making some sort of documentary on what the
average Swede thought about certain political issues such as the
Vietnam War and race issues in the United States. In between asking
politicians and ordinary denizens of Stockholm about their opinions on
politics, she has sex with her drama teacher, classmates, and married
men.<br /><br />What kills me about I AM CURIOUS-YELLOW is that 40
years ago, this was considered pornographic. Really, the sex and
nudity scenes are few and far between, even then it\'s not shot like
some cheaply made porno. While my countrymen mind find it shocking, in
reality sex and nudity are a major staple in Swedish cinema. Even
Ingmar Bergman, arguably their answer to good old boy John Ford, had
sex scenes in his films.<br /><br />I do commend the filmmakers for
the fact that any sex shown in the film is shown for artistic purposes
rather than just to shock people and make money to be shown in
pornographic theaters in America. I AM CURIOUS-YELLOW is a good film
for anyone wanting to study the meat and potatoes (no pun intended) of
Swedish cinema. But really, this film doesn\'t have much of a plot.',
 '"I Am Curious: Yellow" is a risible and pretentious steaming pile.
It doesn\'t matter what one\'s political views are because this film
can hardly be taken seriously on any level. As for the claim that
```

frontal male nudity is an automatic NC-17, that isn\'t true. I\'ve seen R-rated films with male nudity. Granted, they only offer some fleeting views, but where are the R-rated films with gaping vulvas and flapping labia? Nowhere, because they don\'t exist. The same goes for those crappy cable shows: schlongs swinging in the breeze but not a clitoris in sight. And those pretentious indie movies like The Brown Bunny, in which we\'re treated to the site of Vincent Gallo\'s throbbing johnson, but not a trace of pink visible on Chloe Sevigny. Before crying (or implying) "double-standard" in matters of nudity, the mentally obtuse should take into account one unavoidably obvious anatomical difference between men and women: there are no genitals on display when actresses appears nude, and the same cannot be said for a man. In fact, you generally won\'t see female genitals in an American film in anything short of porn or explicit erotica. This alleged double-standard is less a double standard than an admittedly depressing ability to come to terms culturally with the insides of women\'s bodies.',
 "If only to avoid making this type of film in the future. This film is interesting as an experiment but tells no cogent story.

One might feel virtuous for sitting thru it because it touches on so many IMPORTANT issues but it does so without any discernable motive. The viewer comes away with no new perspectives (unless one comes up with one while one's mind wanders, as it will invariably do during this pointless film).

One might better spend one's time staring out a window at a tree growing.

",
 "This film was probably inspired by Godard's Masculin, féminin and I urge you to see that film instead.

The film has two strong elements and those are, (1) the realistic acting (2) the impressive, undeservedly good, photo. Apart from that, what strikes me most is the endless stream of silliness. Lena Nyman has to be most annoying actress in the world. She acts so stupid and with all the nudity in this film,...it's unattractive. Comparing to Godard's film, intellectuality has been replaced with stupidity. Without going too far on this subject, I would say that follows from the difference in ideals between the French and the Swedish society.

A movie of its time, and place. 2/10.",
 'Oh, brother...after hearing about this ridiculous film for umpteen years all I can think of is that old Peggy Lee song..

"Is that all there is??" ...I was just an early teen when this smoked fish hit the U.S. I was too young to get in the theater (although I did manage to sneak into "Goodbye Columbus"). Then a screening at a local film museum beckoned - Finally I could see this film, except now I was as old as my parents were when they schlepped to see it!!

The ONLY reason this film was not condemned to the anonymous sands of time was because of the obscenity case sparked by its U.S. release. MILLIONS of people flocked to this stinker, thinking they were going to see a sex film...Instead, they got lots of closeups of gnarly, repulsive Swedes, on-street interviews in bland shopping malls, asinie political pretension...and feeble who-cares simulated sex scenes with

```
saggy, pale actors.<br /><br />Cultural icon, holy grail, historic artifact..whatever this thing was, shred it, burn it, then stuff the ashes in a lead box!<br /><br />Elite esthetes still scrape to find value in its boring pseudo revolutionary political spewings..But if it weren\'t for the censorship scandal, it would have been ignored, then forgotten.<br /><br />Instead, the "I Am Blank, Blank" rhythymed title was repeated endlessly for years as a titilation for porno films (I am Curious, Lavender - for gay films, I Am Curious, Black - for blaxploitation films, etc..) and every ten years or so the thing rises from the dead, to be viewed by a new generation of suckers who want to see that "naughty sex film" that "revolutionized the film industry"...<br /><br />Yeesh, avoid like the plague..Or if you MUST see it - rent the video and fast forward to the "dirty" parts, just to get it over with.<br /><br />',
 "I would put this at the top of my list of films in the category of unwatchable trash! There are films that are bad, but the worst kind are the ones that are unwatchable but you are suppose to like them because they are supposed to be good for you! The sex sequences, so shocking in its day, couldn't even arouse a rabbit. The so called controversial politics is strictly high school sophomore amateur night Marxism. The film is self-consciously arty in the worst sense of the term. The photography is in a harsh grainy black and white. Some scenes are out of focus or taken from the wrong angle. Even the sound is bad! And some people call this art?<br /><br />",
 "Whoever wrote the screenplay for this movie obviously never consulted any books about Lucille Ball, especially her autobiography. I've never seen so many mistakes in a biopic, ranging from her early years in Celoron and Jamestown to her later years with Desi. I could write a whole list of factual errors, but it would go on for pages. In all, I believe that Lucille Ball is one of those inimitable people who simply cannot be portrayed by anyone other than themselves. If I were Lucie Arnaz and Desi, Jr., I would be irate at how many mistakes were made in this film. The filmmakers tried hard, but the movie seems awfully sloppy to me."]

sentences = []
word_threshold = 32

# Получить отдельные предложения и поместить их в sentences
for sentence in tqdm(dataset['train']['text']):
    sentences.extend(
        [x.lower() for x in sent_tokenize(sentence) if
len(word_tokenize(x.lower())) < word_threshold]
    )

{"model_id":"ecaf1a323c57421d8e6d74e531348658","version_major":2,"vers
ion_minor":0}

print("Всего предложений:", len(sentences))
```

```
Всего предложений: 198801
```

Посчитаем для каждого слова его встречаемость.

```
words = Counter()

# Расчет встречаемости слов
for sentence in tqdm(sentences):
    for word in word_tokenize(sentence):
        words[word] += 1

{"model_id":"5a11b338a5714bfb92ac8e694883145c","version_major":2,"vers
ion_minor":0}

len(words)

68415
```

Добавим в словарь `vocab_size` самых встречающихся слов.

```
vocab = set()
# vocab_size = 40000
vocab_size = 20000

# Наполнение словаря
vocab = set(['<unk>', '<bos>', '<eos>', '<pad>'])

# Топ слов (длиной равной vocab_size)
words_most_common = [word for word, _ in
words.most_common(vocab_size)]
vocab.update(words_most_common)

assert '<unk>' in vocab
assert '<bos>' in vocab
assert '<eos>' in vocab
assert '<pad>' in vocab
assert len(vocab) == vocab_size + 4

print("Всего слов в словаре:", len(vocab))

Всего слов в словаре: 20004
```

Подготовка датасета (1 балл)

Далее, как и в семинарском занятии, подготовим датасеты и даталоадеры.

В классе `WordDataset` вам необходимо реализовать метод `__getitem__`, который будет возвращать сэмпл данных по входному idx, то есть список целых чисел (индексов слов).

Внутри этого метода необходимо добавить служебные токены начала и конца последовательности, а также токенизировать соответствующее предложение с помощью `word_tokenize` и сопоставить ему индексы из `word2ind`.

```
word2ind = {char: i for i, char in enumerate(vocab)}
ind2word = {i: char for char, i in word2ind.items()}

class WordDataset:
    def __init__(self, sentences):
        self.data = sentences
        self.unk_id = word2ind['<unk>']
        self.bos_id = word2ind['<bos>']
        self.eos_id = word2ind['<eos>']
        self.pad_id = word2ind['<pad>']

    def __getitem__(self, idx: int) -> List[int]:
        tokenized_sentence = []
        # Допишите код здесь
        tokenized_sentence += [self.bos_id]
        tokenized_sentence += [word2ind.get(word, self.unk_id) for
word in word_tokenize(self.data[idx])]
        tokenized_sentence += [self.eos_id]

        return tokenized_sentence

    def __len__(self) -> int:
        return len(self.data)

def collate_fn_with_padding(
    input_batch: List[List[int]], pad_id=word2ind['<pad>']) ->
torch.Tensor:
    seq_lens = [len(x) for x in input_batch]
    max_seq_len = max(seq_lens)

    new_batch = []
    for sequence in input_batch:
        for _ in range(max_seq_len - len(sequence)):
            sequence.append(pad_id)
        new_batch.append(sequence)

    sequences = torch.LongTensor(new_batch).to(device)

    new_batch = {
        'input_ids': sequences[:,:-1],
        'target_ids': sequences[:,1:]
    }

    return new_batch

train_sentences, temp_sentences = train_test_split(sentences,
test_size=0.2, random_state=42)
```

```
val_sentences, test_sentences = train_test_split(temp_sentences,
test_size=0.5, random_state=42)

train_dataset = WordDataset(train_sentences)
val_dataset = WordDataset(val_sentences)
test_dataset = WordDataset(test_sentences)

batch_size = 128

train_dataloader = DataLoader(
    train_dataset, collate_fn=collate_fn_with_padding,
batch_size=batch_size)

val_dataloader = DataLoader(
    val_dataset, collate_fn=collate_fn_with_padding,
batch_size=batch_size)

test_dataloader = DataLoader(
    test_dataset, collate_fn=collate_fn_with_padding,
batch_size=batch_size)
```

Обучение и архитектура модели

Вам необходимо на практике проверить, что влияет на качество языковых моделей. В этом задании нужно провести серию экспериментов с различными вариантами языковых моделей и сравнить различия в конечной перплексии на тестовом множестве.

Возмоэные идеи для экспериментов:

- Различные RNN-блоки, например, LSTM или GRU. Также можно добавить сразу несколько RNN блоков друг над другом с помощью аргумента num_layers. Вам поможет официальная документация здесь
- Различные размеры скрытого состояния. Различное количество линейных слоев после RNN-блока. Различные функции активации.
- Добавление нормализаций в виде Dropout, BatchNorm или LayerNorm
- Различные аргументы для оптимизации, например, подбор оптимального learning rate или тип алгоритма оптимизации SGD, Adam, RMSProp и другие
- Любые другие идеи и подходы

После проведения экспериментов необходимо составить таблицу результатов, в которой описан каждый эксперимент и посчитана перплексия на тестовом множестве.

Учтите, что эксперименты, которые различаются, например, только размером скрытого состояния или количеством линейных слоев считаются, как один эксперимент.

Успехов!

Функция evaluate (1 балл)

Заполните функцию `evaluate`

```
def evaluate(model, criterion, dataloader) -> float:
    model.eval()
    losses = []
    with torch.no_grad():
        for batch in dataloader:
            logits = model(batch['input_ids']).flatten(start_dim=0,
end_dim=1)
            loss = criterion(
                logits,
                batch['target_ids'].flatten()
            )
            losses.append(loss.item())

    mean_loss = sum(losses) / len(losses)
    perplexity = math.exp(mean_loss)

    return mean_loss, perplexity
```

Train loop (1 балл)

Напишите функцию для обучения модели.

```
import pandas as pd

# Датасет для сбора статистик

df_stats = pd.DataFrame(columns=['Model', 'Loss', 'Perplexity',
'Diff_Loss', 'Diff_Perplexity', 'Better_than_base'])

display(df_stats)
```

```
Empty DataFrame
Columns: [Model, Loss, Perplexity, Diff_Loss, Diff_Perplexity,
Better_than_base]
Index: []
```

```
from IPython.display import clear_output

def train_model(model, criterion, optimizer,
                model_name='Current',
                plot=True,
                num_epoch=5,
                base_losses=None,
                base_perplexities=None,
                train_loader=train_dataloader,
                val_loader=val_dataloader,
                stop_epoch=3,
                stop_delta=0.01):

    # Напишите код здесь
```

```
    train_losses = []
    val_losses = []
    train_perps = []
    val_perps = []

    # perplexities = []

    # best_perp = float('inf')
    best_loss = float('inf')
    n_stop_epoch = 0
    best_state = None

    for epoch in range(num_epoch):
        epoch_losses = []
        epoch_perps = []
        model.train()

        for batch in tqdm(train_loader, desc=f'Training epoch
{epoch}'):
            optimizer.zero_grad()
            logits = model(batch['input_ids']).flatten(start_dim=0,
end_dim=1)
            loss = criterion(logits, batch['target_ids'].flatten())
            loss.backward()

            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)

            optimizer.step()

            epoch_losses.append(loss.item())
            epoch_perps.append(math.exp(loss.item()))

        train_loss = sum(epoch_losses) / len(epoch_losses)
        train_perp = math.exp(train_loss)
        val_loss, val_perp = evaluate(model, criterion, val_loader)

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_perps.append(train_perp)
        val_perps.append(val_perp)

        # # Условие ранней остановки
        # if val_perp < best_perp - stop_delta:
        #     best_perp = val_perp
        #     best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}
        #     n_stop_epoch = 0
        # else:
        #     n_stop_epoch += 1
```

```
        # Условие ранней остановки
        if val_loss < best_loss - stop_delta:
            best_loss = val_loss
            best_state = {k: v.cpu().clone() for k, v in
model.state_dict().items()}
            n_stop_epoch = 0
        else:
            n_stop_epoch += 1

        # Отрисовка графиков
        if plot:
            fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(15,
6))

            # Loss
            if model_name != 'GRU_Base_Model' and base_losses is not
None:
                axes[0].plot(
                    range(1, len(base_losses) + 1), base_losses,
                    color='black', linestyle='--', linewidth=2,
                    marker='o', markersize=4, label='GRU_Base_Model
(Val Loss)'
                )

            # Train Loss
            axes[0].plot(
                range(1, len(train_losses) + 1), train_losses,
                color='blue', marker='s', markersize=4, linewidth=2,
                label=f'{model_name} (Train Loss)'
            )

            # Val Loss
            axes[0].plot(
                range(1, len(val_losses) + 1), val_losses,
                color='red', marker='o', markersize=4, linewidth=2,
                label=f'{model_name} (Val Loss)'
            )

            axes[0].set_title('Losses')
            axes[0].set_xlabel('epoch')
            axes[0].set_ylabel('loss')
            axes[0].grid(True, alpha=0.3)
            axes[0].legend()

            # Perplexity
            if model_name != 'GRU_Base_Model' and base_perplexities is
not None:
                axes[1].plot(
                    range(1, len(base_perplexities) + 1),
```

```
base_perplexities,
                    color='black', linestyle='--', linewidth=2,
                    marker='o', markersize=4, label='GRU_Base_Model
(Val Perplexity)'
                )

            # Train Perplexity
            train_perps_correct = [math.exp(l) for l in train_losses]
            axes[1].plot(
                range(1, len(train_perps_correct) + 1),
train_perps_correct,
                color='blue', marker='s', markersize=4, linewidth=2,
                label=f'{model_name} (Train Perplexity)'
            )

            # Val Perplexity
            axes[1].plot(
                range(1, len(val_perps) + 1), val_perps,
                color='red', marker='o', markersize=4, linewidth=2,
                label=f'{model_name} (Val Perplexity)'
            )

            axes[1].set_title('Perplexities')
            axes[1].set_xlabel('epoch')
            axes[1].set_ylabel('perplexity')
            axes[1].grid(True, alpha=0.3)
            axes[1].legend()

            clear_output(wait=True)
            plt.tight_layout()
            plt.show()
            plt.close(fig)

        # Ранняя остановка
        if n_stop_epoch >= stop_epoch:
            break

    # Загрузка модели с лучшими весами
    model.load_state_dict(best_state)

    # Заполнение таблицы
    best_idx = int(np.argmin(val_perps))
    curr_model_loss = val_losses[best_idx]
    curr_model_perp = val_perps[best_idx]

    if model_name != 'GRU_Base_Model' and base_perplexities is not
None:
        best_base_idx = int(np.argmin(base_perplexities))
        best_base_loss = base_losses[best_base_idx]
        best_base_perp = base_perplexities[best_base_idx]
```

```
    else:
        best_base_loss = 0.0
        best_base_perp = 0.0

    if model_name != 'GRU_Base_Model':
        diff_loss = curr_model_loss - best_base_loss
        diff_perp = curr_model_perp - best_base_perp
        better_than_base = curr_model_perp < best_base_perp
    else:
        diff_loss = 0
        diff_perp = 0
        better_than_base = False

    data_to_add = (model_name, curr_model_loss, curr_model_perp,
                   diff_loss, diff_perp, better_than_base)
    if model_name in df_stats['Model'].values:
        idx = df_stats[df_stats['Model'] == model_name].index
    else:
        idx = len(df_stats)
    df_stats.loc[idx] = data_to_add

    print(f'Loss = {curr_model_loss}')
    print(f'Perplexity = {curr_model_perp}')

    # Сравнение текущей модели с базовой моделью
    if model_name != 'GRU_Base_Model':
        print('\nСравнение с GRU_Base_Model:')
        print('Loss', ('Хуже', 'Лучше')[curr_model_loss <
best_base_loss], 'на', abs(diff_loss))
        print('Perplexity', ('Хуже', 'Лучше')[curr_model_perp <
best_base_perp], 'на', abs(diff_perp))

    return val_losses, val_perps
```

Base model

В качестве базовой модели возьмем модель из семинара. Результаты моделей из дальнейших экспериментов будем сравнивать с базовой моделью. Оценка того, какая из моделей лучше, будет основываться на метрике Perplexity.

```
class GRU_Base_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
```

```
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)  # [batch_size,
seq_len, hidden_dim]
        output, _ = self.rnn(embeddings)  # [batch_size, seq_len,
hidden_dim]
        output = self.dropout(self.linear(self.non_lin(output)))  #
[batch_size, seq_len, hidden_dim]
        projection = self.projection(self.non_lin(output))  #
[batch_size, seq_len, vocab_size]

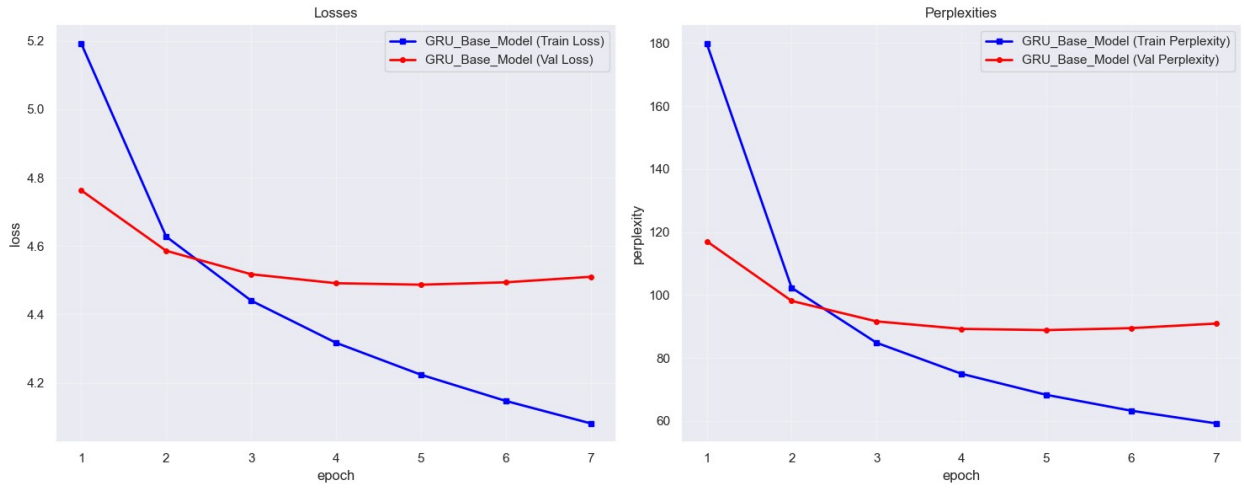
        return projection

set_seed(SEED)

model_name = 'GRU_Base_Model'

model_gru_base = GRU_Base_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_gru_base.parameters())

base_losses, base_perplexities = train_model(model=model_gru_base,
                                             criterion=criterion,
                                             optimizer=optimizer,
                                             model_name=model_name,
                                             num_epoch=NUM_EPOCH)
```




```
Loss = 4.486887384683658
Perplexity = 88.84447637457775

display(df_stats)
```

```
            Model      Loss  Perplexity  Diff_Loss  Diff_Perplexity  \
0  GRU_Base_Model  4.486887   88.844476          0                0

   Better_than_base
0             False
```

Замена GRU на LSTM (Первый эксперимент (2 балла))

Определите архитектуру модели и обучите её.

LSTM в отличии GRU лучше запоминает длинные зависимости.

```
class LSTM_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

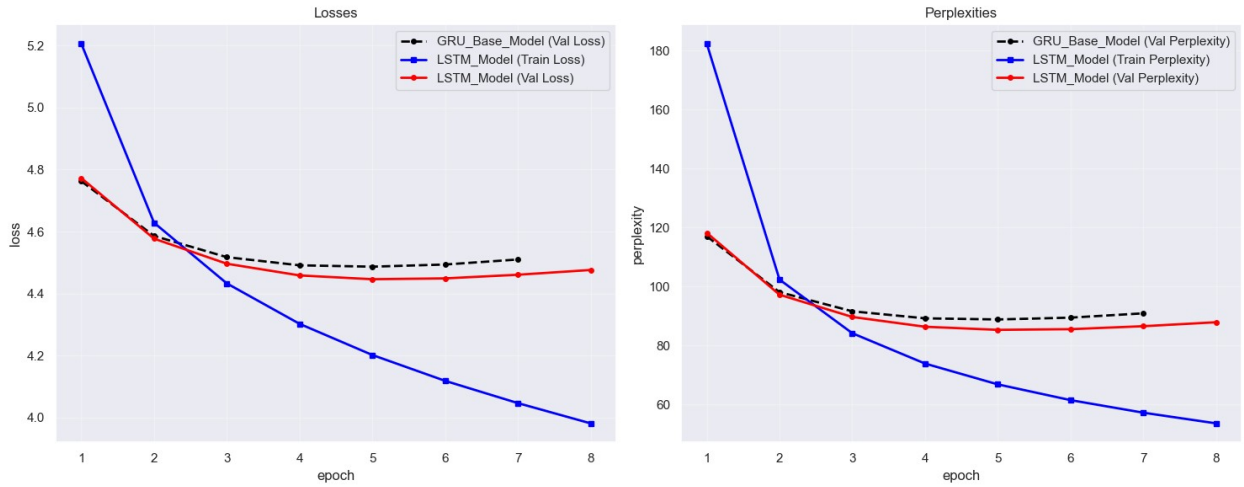
set_seed(SEED)

model_name = 'LSTM_Model'

model_lstm = LSTM_LM(hidden_dim=256, vocab_size=len(vocab),
num_layers=1).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_lstm.parameters())

lstm_losses, lstm_perplexities = train_model(model=model_lstm,
                                             criterion=criterion,
                                             optimizer=optimizer,
                                             model_name=model_name,
                                             plot=True,
                                             num_epoch=NUM_EPOCH,
                                             base_losses=base_losses,
```

```
base_perplexities=base_perplexities)
```




```
Loss = 4.4464058723205175
Perplexity = 85.31974222455521

Сравнение с GRU_Base_Model:
Loss Лучше на 0.040481512363140126
Perplexity Лучше на 3.524734150022539

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True

Использование сразу нескольких RNN блоков (Второй эксперимент (2 балла))

Попробуйте что-то поменять в модели или в пайплайне обучения, идеи для экспериментов можно подсмотреть выше.

Два слоя в RNN num_layers = 2. Позволит выделить более сложные иерархические зависимости.

```
class MultiLayerGRU_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
```

```
        self.rnn = nn.GRU(hidden_dim, hidden_dim,
num_layers=num_layers, batch_first=True, dropout=0.1)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'MultiLayer_GRU_Model'

model_multilayer_gru = MultiLayerGRU_LM(hidden_dim=256,
vocab_size=len(vocab), num_layers=2).to(device) # num_layers=2
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_multilayer_gru.parameters())

multilayer_losses, multilayer_perplexities =
train_model(model=model_multilayer_gru,

criterion=criterion,

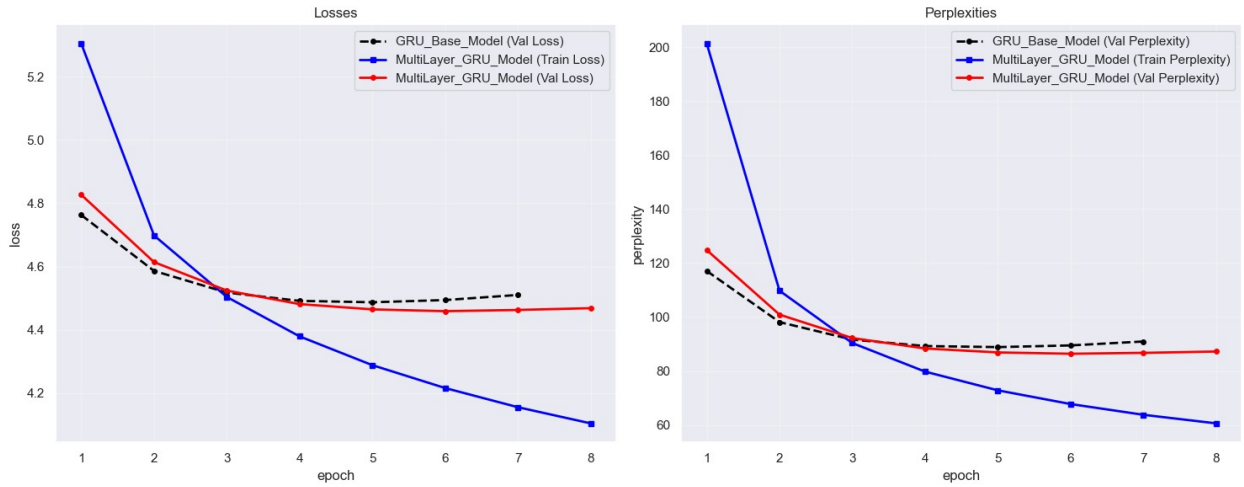
optimizer=optimizer,

model_name=model_name,
                                                        plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.4586478013258715
Perplexity = 86.37063983423866

Сравнение с GRU_Base_Model:
Loss Лучше на 0.028239583357786202
Perplexity Лучше на 2.473836540339093

display(df_stats)

                   Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0        GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1           LSTM_Model  4.446406   85.319742  -0.040482          -
3.524734
2  MultiLayer_GRU_Model  4.458648   86.370640  -0.028240          -
2.473837

   Better_than_base
0             False
1              True
2              True
```

Два независимых блока GRU с dropout между ними.

```python
class Stacked_GRU_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn1 = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.rnn2 = nn.GRU(hidden_dim, hidden_dim, batch_first=True)

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)
```

```
        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn1(embeddings)
        output = self.dropout(output)
        output, _ = self.rnn2(output)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Stacked_GRU_Model'

model_multilayer_gru = Stacked_GRU_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_multilayer_gru.parameters())

stacked_gru_losses, stacked_gru_perplexities =
train_model(model=model_multilayer_gru,

criterion=criterion,

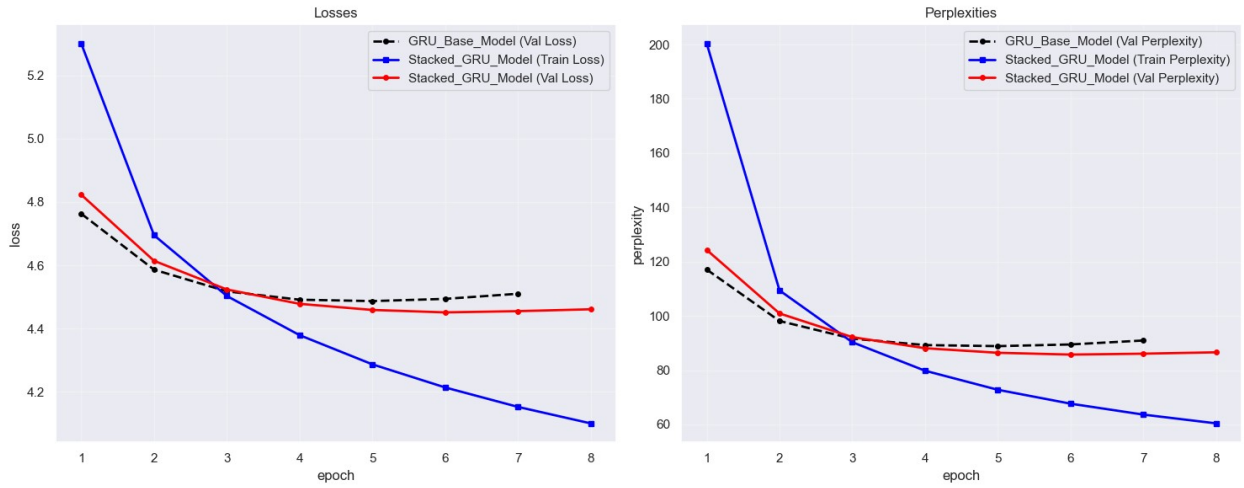
optimizer=optimizer,

model_name=model_name,
                                                        plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.451247099118355
Perplexity = 85.73379590217976

Сравнение с GRU_Base_Model:
Loss Лучше на 0.035640285565302854
Perplexity Лучше на 3.1106804723979877

display(df_stats)

                    Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0         GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1            LSTM_Model  4.446406   85.319742  -0.040482         -
3.524734
2  MultiLayer_GRU_Model  4.458648   86.370640  -0.028240         -
2.473837
3     Stacked_GRU_Model  4.451247   85.733796  -0.035640         -
3.110680

   Better_than_base
0             False
1              True
2              True
3              True
```

Далее будут ещё эксперименты

Увеличенние размера скрытого состояния

hidden_dim=512 (и embedding_dim=512) позволит изучить более сложные закономерности.

```
class LargerHiddenDim_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'LargerHiddenDim_Model'

model_larger_hidden = LargerHiddenDim_LM(hidden_dim=512,
vocab_size=len(vocab)).to(device) # hidden_dim=512
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_larger_hidden.parameters())

larger_hidden_losses, larger_hidden_perplexities =
train_model(model=model_larger_hidden,

criterion=criterion,

optimizer=optimizer,

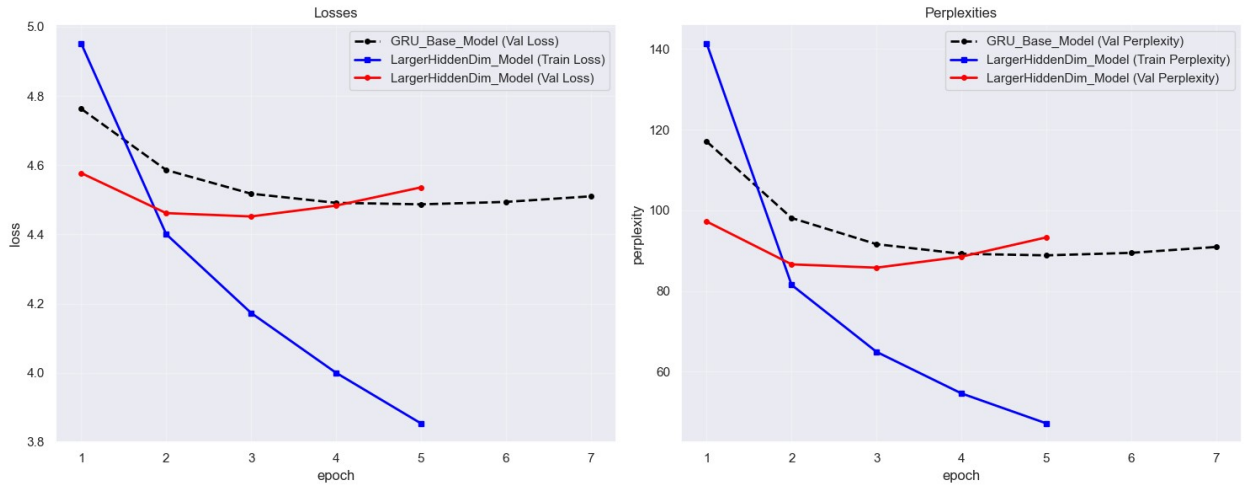
model_name=model_name,

plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.451895860525278
Perplexity = 85.7894347264396

Сравнение с GRU_Base_Model:
Loss Лучше на 0.03499152415837958
Perplexity Лучше на 3.0550416481381575

display(df_stats)

                    Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0         GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1             LSTM_Model  4.446406   85.319742  -0.040482         -
3.524734
2   MultiLayer_GRU_Model  4.458648   86.370640  -0.028240         -
2.473837
3      Stacked_GRU_Model  4.451247   85.733796  -0.035640         -
3.110680
4  LargerHiddenDim_Model  4.451896   85.789435  -0.034992         -
3.055042

   Better_than_base
0             False
1              True
2              True
3              True
4              True
```

Увеличенние количества линейных слоёв после RNN

Модель с двумя дополнительными линейными слоями. Теоретически позволит лучше распредлять вероятность следования слов.

```
class LargerLinearLayers_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)

        # Два линейных слоя
        self.linear_layers = nn.Sequential(
            nn.Linear(hidden_dim, 2 * hidden_dim),
            nn.Tanh(),
            nn.Dropout(p=0.1),
            nn.Linear(2 * hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Dropout(p=0.1)
        )
        self.non_lin = nn.Tanh()
        self.projection = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.linear_layers(output)
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'LargerLinearLayers_Model'

model_larger_linear = LargerLinearLayers_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_larger_linear.parameters())

larger_linear_losses, larger_linear_perplexities =
train_model(model=model_larger_linear,

criterion=criterion,

optimizer=optimizer,

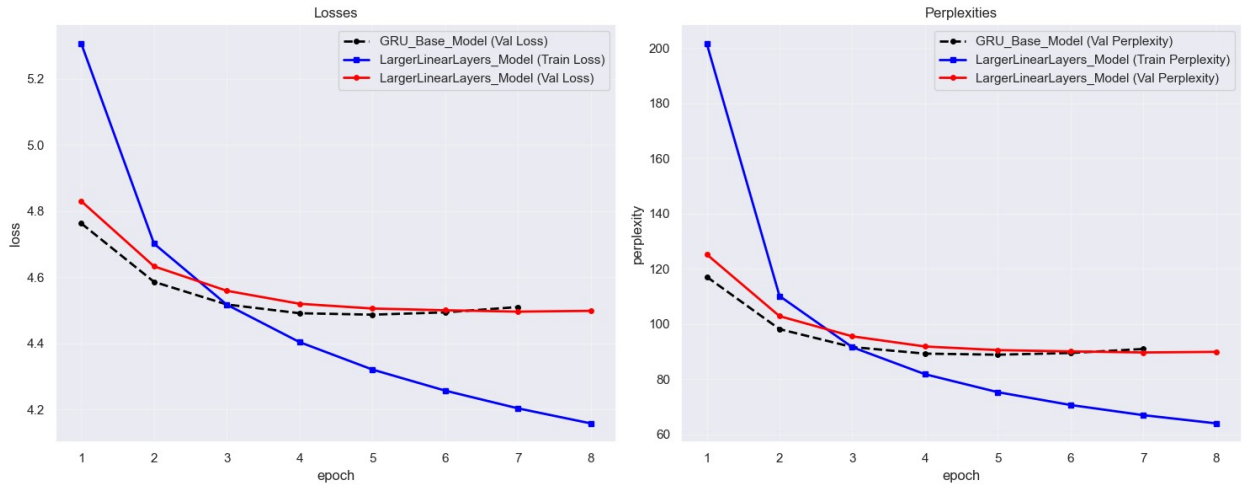
model_name=model_name,

plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.496023282026633
Perplexity = 89.65986939231394

Сравнение с GRU_Base_Model:
Loss Хуже на 0.009135897342975596
Perplexity Хуже на 0.8153930177361843

display(df_stats)

                      Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0           GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1              LSTM_Model  4.446406   85.319742  -0.040482         -
3.524734
2     MultiLayer_GRU_Model  4.458648   86.370640  -0.028240         -
2.473837
3        Stacked_GRU_Model  4.451247   85.733796  -0.035640         -
3.110680
4    LargerHiddenDim_Model  4.451896   85.789435  -0.034992         -
3.055042
5  LargerLinearLayers_Model  4.496023   89.659869   0.009136
0.815393

   Better_than_base
0             False
1              True
2              True
3              True
4              True
5             False
```

Использование альтернативной функции активации (GELU)

Применим функцию активации GELU вместо Tanh. Gaussian Error Linear Unit учитывает вероятностный характер нейронов.

```
class GELU_GRU_LM(nn.Module):

    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()    # GELU вместо Tanh
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

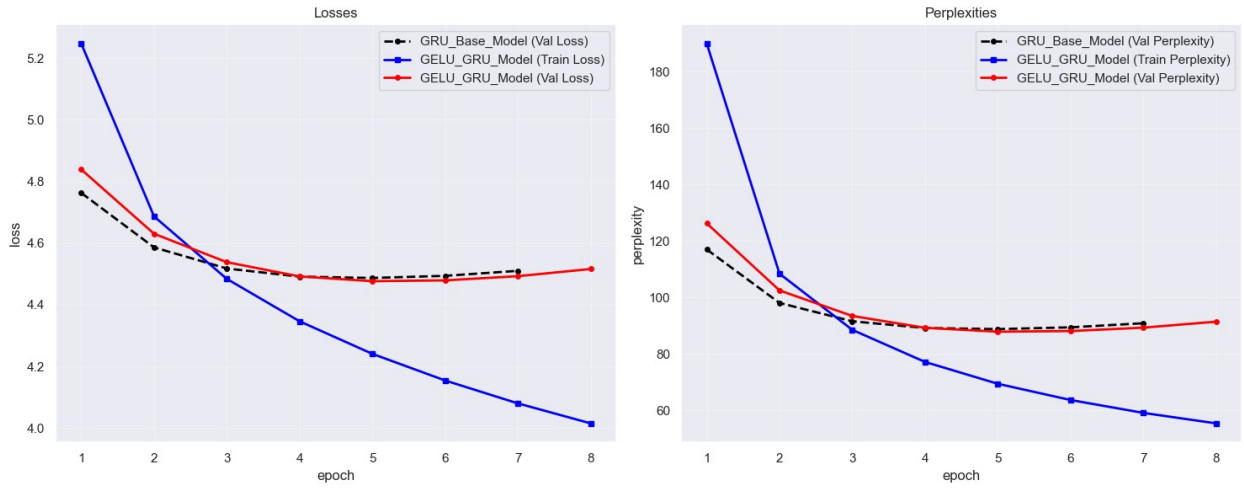
set_seed(SEED)

model_name = 'GELU_GRU_Model'

model_gelu = GELU_GRU_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_gelu.parameters())

gelu_losses, gelu_perplexities = train_model(model=model_gelu,
                                             criterion=criterion,
                                             optimizer=optimizer,
                                             model_name=model_name,
                                             plot=True,
                                             num_epoch=NUM_EPOCH,
                                             base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.476464772835756
Perplexity = 87.92329378583186

Сравнение с GRU_Base_Model:
Loss Лучше на 0.010422611847901386
Perplexity Лучше на 0.9211825887458929

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183

	Better_than_base
0	False
1	True
2	True
3	True
4	True
5	False
6	True

Применение различных нормализаций

Увеличенние Dropout в комбинации с двумя дполнительными линейными слоями для регуляризации. Попробуем dropout = 0.3, тем самым уменьшим вероятность переобучения.

```
class LargerDropout_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, dropout_rate:
float = 0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)

        # Два линейных слоя и dropout_rate
        self.linear_layers = nn.Sequential(
            nn.Linear(hidden_dim, 2 * hidden_dim),
            nn.Tanh(),
            nn.Dropout(p=dropout_rate),
            nn.Linear(2 * hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Dropout(p=dropout_rate)
        )
        self.non_lin = nn.Tanh()
        self.projection = nn.Linear(hidden_dim, vocab_size)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.linear_layers(output)
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'LargerDropout_Model'

model_larger_dropout = LargerDropout_LM(hidden_dim=256,
vocab_size=len(vocab), dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_larger_dropout.parameters())

larger_dropout_losses, larger_dropout_perplexities =
train_model(model=model_larger_dropout,

criterion=criterion,

optimizer=optimizer,

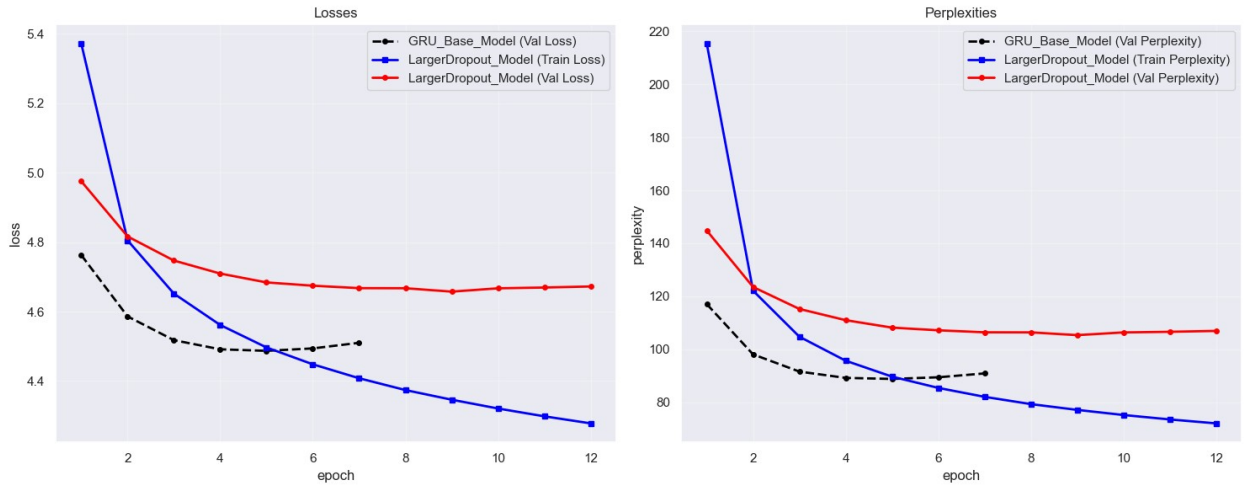
model_name=model_name,
```

```
plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.657434729429392
Perplexity = 105.36544430198418

Сравнение с GRU_Base_Model:
Loss Хуже на 0.1705473447457342
Perplexity Хуже на 16.52096792740643

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183
7	LargerDropout_Model	4.657435	105.365444	0.170547	

```
16.520968

   Better_than_base
0             False
1              True
2              True
3              True
4              True
5             False
6              True
7             False
```

Добавление BatchNorm1d после RNN.

```
class BatchNorm_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        output = output.transpose(1, 2)
        output = self.batch_norm(output)
        output = output.transpose(1, 2)

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))
        return projection

set_seed(SEED)

model_name = 'BatchNorm_Model'

model_batchnorm = BatchNorm_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_batchnorm.parameters())

batchnorm_losses, batchnorm_perplexities =
train_model(model=model_batchnorm,
```

```
criterion=criterion,

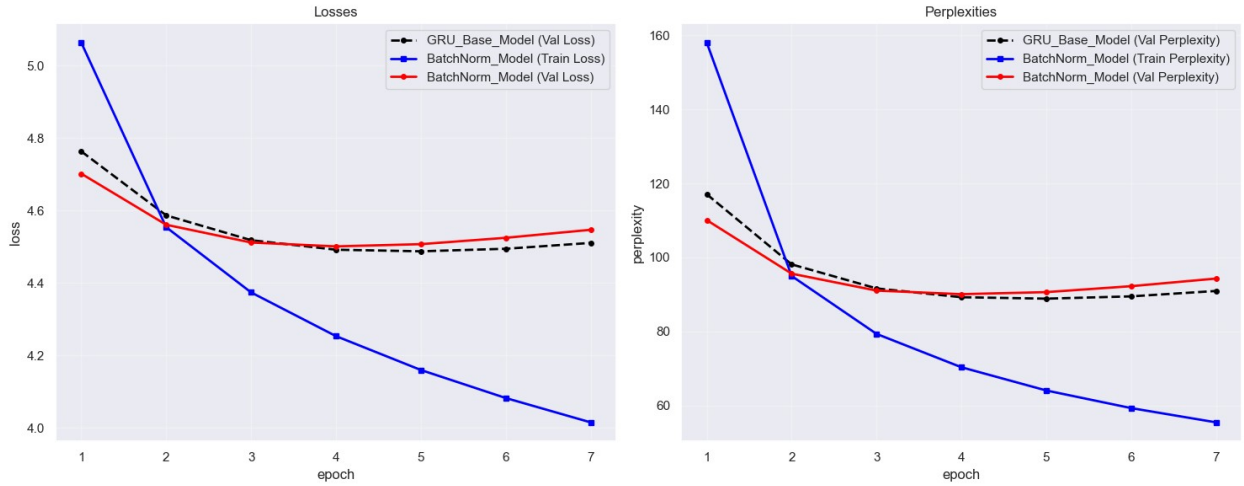
optimizer=optimizer,

model_name=model_name,
                                                        plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.50042196114858
Perplexity = 90.05512304759414

Сравнение с GRU_Base_Model:
Loss Хуже на 0.013534576464921955
Perplexity Хуже на 1.2106466730163845

display(df_stats)

                       Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0            GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1               LSTM_Model  4.446406   85.319742  -0.040482        -
3.524734
2      MultiLayer_GRU_Model  4.458648   86.370640  -0.028240        -
2.473837
3         Stacked_GRU_Model  4.451247   85.733796  -0.035640        -
3.110680
4     LargerHiddenDim_Model  4.451896   85.789435  -0.034992        -
3.055042
5  LargerLinearLayers_Model  4.496023   89.659869   0.009136
```

```
0.815393
6            GELU_GRU_Model  4.476465   87.923294  -0.010423        -
0.921183
7       LargerDropout_Model  4.657435  105.365444   0.170547
16.520968
8           BatchNorm_Model  4.500422   90.055123   0.013535
1.210647

   Better_than_base
0             False
1              True
2              True
3              True
4              True
5             False
6              True
7             False
8             False
```

Добавление LayerNorm после RNN. Предпочтительнее для моделей RNN, нежели BatchNorm.

```
class LayerNorm_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.GRU(hidden_dim, hidden_dim, batch_first=True)
        self.layer_norm = nn.LayerNorm(hidden_dim)  # LayerNorm
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.layer_norm(output)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'LayerNorm_Model'

model_layernorm = LayerNorm_LM(hidden_dim=256,
vocab_size=len(vocab)).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
```

```
optimizer = torch.optim.Adam(model_layernorm.parameters())

layernorm_losses, layernorm_perplexities =
train_model(model=model_layernorm,

criterion=criterion,

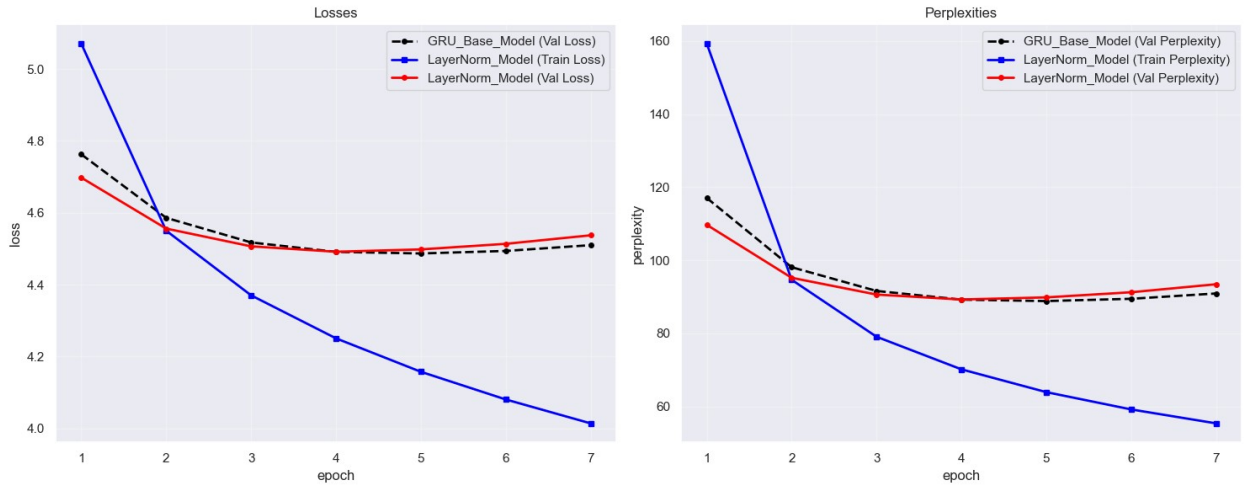
optimizer=optimizer,

model_name=model_name,
                                                        plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.491675493044731
Perplexity = 89.27089340554895

Сравнение с GRU_Base_Model:
Loss Хуже на 0.004788108361073462
Perplexity Хуже на 0.4264170309711943

display(df_stats)

                     Model      Loss  Perplexity  Diff_Loss
Diff_Perplexity  \
0           GRU_Base_Model  4.486887   88.844476   0.000000
0.000000
1               LSTM_Model  4.446406   85.319742  -0.040482        -
3.524734
2      MultiLayer_GRU_Model  4.458648   86.370640  -0.028240        -
2.473837
```

```
3         Stacked_GRU_Model  4.451247   85.733796  -0.035640        -
3.110680
4     LargerHiddenDim_Model  4.451896   85.789435  -0.034992        -
3.055042
5  LargerLinearLayers_Model  4.496023   89.659869   0.009136
0.815393
6            GELU_GRU_Model  4.476465   87.923294  -0.010423        -
0.921183
7       LargerDropout_Model  4.657435  105.365444   0.170547
16.520968
8           BatchNorm_Model  4.500422   90.055123   0.013535
1.210647
9           LayerNorm_Model  4.491675   89.270893   0.004788
0.426417

   Better_than_base
0             False
1              True
2              True
3              True
4              True
5             False
6              True
7             False
8             False
9             False
```

Увеличенние размера эмбеддингов

embedding_dim=512 для увеличения векотора предствления каждого слова. Для каждого слова кодируется больше информации.

```
class LargerEmbeddingDim_LM(nn.Module):
    def __init__(self, embedding_dim: int, hidden_dim: int,
vocab_size: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.rnn = nn.GRU(embedding_dim, hidden_dim, batch_first=True)
        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=0.1)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)
        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))
```

```
        return projection

set_seed(SEED)

model_name = 'LargerEmbeddingDim_Model'

model_larger_emb = LargerEmbeddingDim_LM(embedding_dim=512,
hidden_dim=256, vocab_size=len(vocab)).to(device) # embedding_dim=512
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(model_larger_emb.parameters())

larger_emb_losses, larger_emb_perplexities =
train_model(model=model_larger_emb,

criterion=criterion,

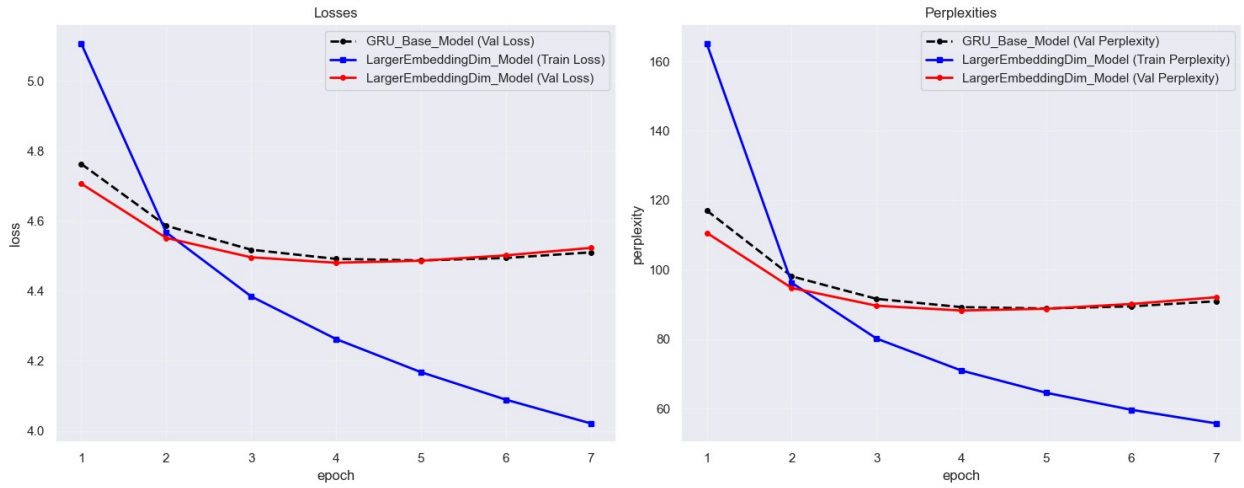
optimizer=optimizer,

model_name=model_name,
                                                        plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.480217933654785
Perplexity = 88.253904075855

Сравнение с GRU_Base_Model:
Loss Лучше на 0.006669451028872508
Perplexity Лучше на 0.5905722987227477
```

```
display(df_stats)
```

```
                        Model      Loss  Perplexity  Diff_Loss  \
0             GRU_Base_Model  4.486887   88.844476   0.000000
1                 LSTM_Model  4.446406   85.319742  -0.040482
2       MultiLayer_GRU_Model  4.458648   86.370640  -0.028240
3          Stacked_GRU_Model  4.451247   85.733796  -0.035640
4      LargerHiddenDim_Model  4.451896   85.789435  -0.034992
5   LargerLinearLayers_Model  4.496023   89.659869   0.009136
6             GELU_GRU_Model  4.476465   87.923294  -0.010423
7        LargerDropout_Model  4.657435  105.365444   0.170547
8            BatchNorm_Model  4.500422   90.055123   0.013535
9            LayerNorm_Model  4.491675   89.270893   0.004788
10  LargerEmbeddingDim_Model  4.480218   88.253904  -0.006669

    Diff_Perplexity  Better_than_base
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
```

Финальная модель (комбинирование подходов)

Хорошо показали себя подходы:

- Замена GRU на LSTM
- Использование сразу нескольких RNN блоков
- Увеличение размера скрытого состояния
- Применения нормализаций BatchNom и LayerNorm (выберем LayerNorm, потому что данная нормализация является лучшей практикой для RNN моделей)

Применим подходы в финальной модели:

```
class Combined_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 1, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)
```

```
        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_Model'

combined_model = Combined_LM(hidden_dim=512, vocab_size=len(vocab),
num_layers=2).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_model.parameters())
# optimizer = torch.optim.AdamW(combined_model.parameters(), lr=0.001,
weight_decay=0.01)

combined_losses, combined_perplexities =
train_model(model=combined_model,

criterion=criterion,

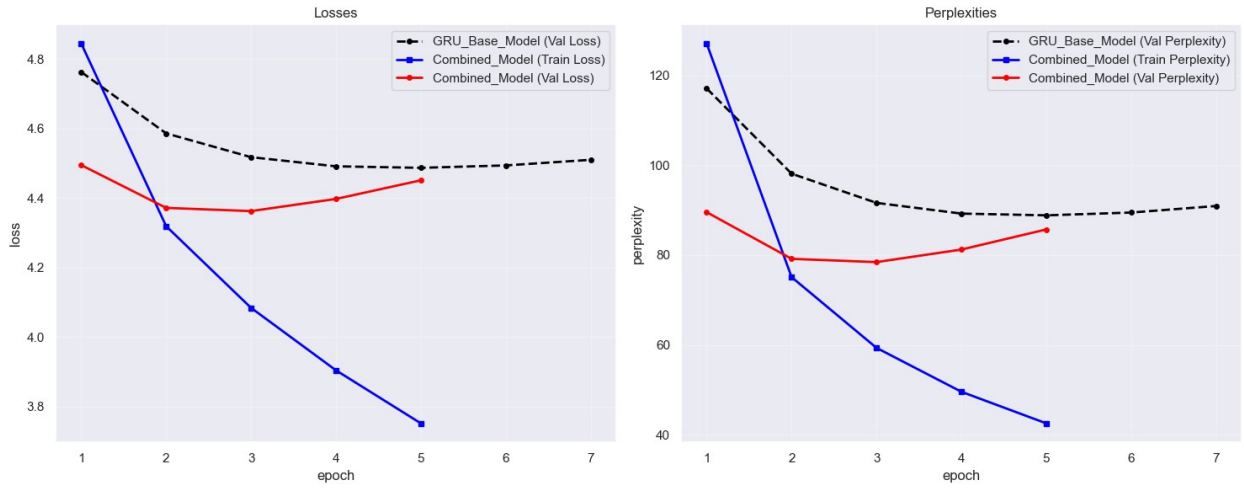
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.3624130212343655
Perplexity = 78.44619856634573

Сравнение с GRU_Base_Model:
Loss Лучше на 0.12447436344929219
Perplexity Лучше на 10.398277808232024

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669		
11	Combined_Model	4.362413	78.446199	-0.124474		

```
10        -0.590572              True
11       -10.398278              True
```

Вариант с GELU

```
class Combined_GELU_LM(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 1, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model'

combined_gelu_model = Combined_GELU_LM(hidden_dim=512,
vocab_size=len(vocab), num_layers=2).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_gelu_model.parameters())
# optimizer = torch.optim.AdamW(combined_gelu_model.parameters(),
lr=0.001, weight_decay=0.01)
```

```
combined_gelu_losses, combined_gelu_perplexities =
train_model(model=combined_gelu_model,

criterion=criterion,

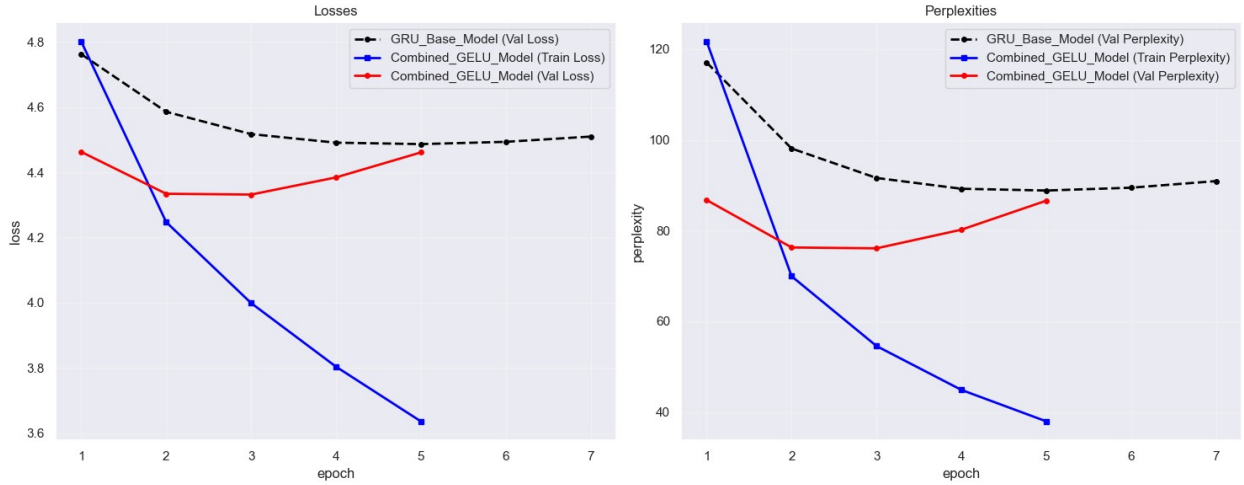
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.331979864682907
Perplexity = 76.09479492395792

Сравнение с GRU_Base_Model:
Loss Лучше на 0.1549075200007506
Perplexity Лучше на 12.749681450619832

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss
0	GRU_Base_Model	4.486887	88.844476	0.000000
1	LSTM_Model	4.446406	85.319742	-0.040482
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136
6	GELU_GRU_Model	4.476465	87.923294	-0.010423
7	LargerDropout_Model	4.657435	105.365444	0.170547
8	BatchNorm_Model	4.500422	90.055123	0.013535

```
9            LayerNorm_Model  4.491675   89.270893   0.004788
10  LargerEmbeddingDim_Model  4.480218   88.253904  -0.006669
11           Combined_Model  4.362413   78.446199  -0.124474
12       Combined_GELU_Model  4.331980   76.094795  -0.154908
```

```
    Diff_Perplexity  Better_than_base
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
11       -10.398278              True
12       -12.749681              True
```

Попробуем еще варианты комбинированных моделей

Увеличенный дропаут: dropout_rate=0.3

```
class Combined_LM2(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm
```

```
        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_Model2'

combined_model2 = Combined_LM2(hidden_dim=512, vocab_size=len(vocab),
num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_model2.parameters())
# optimizer = torch.optim.AdamW(combined_model.parameters(), lr=0.001,
weight_decay=0.01)

combined_losses2, combined_perplexities2 =
train_model(model=combined_model2,

criterion=criterion,

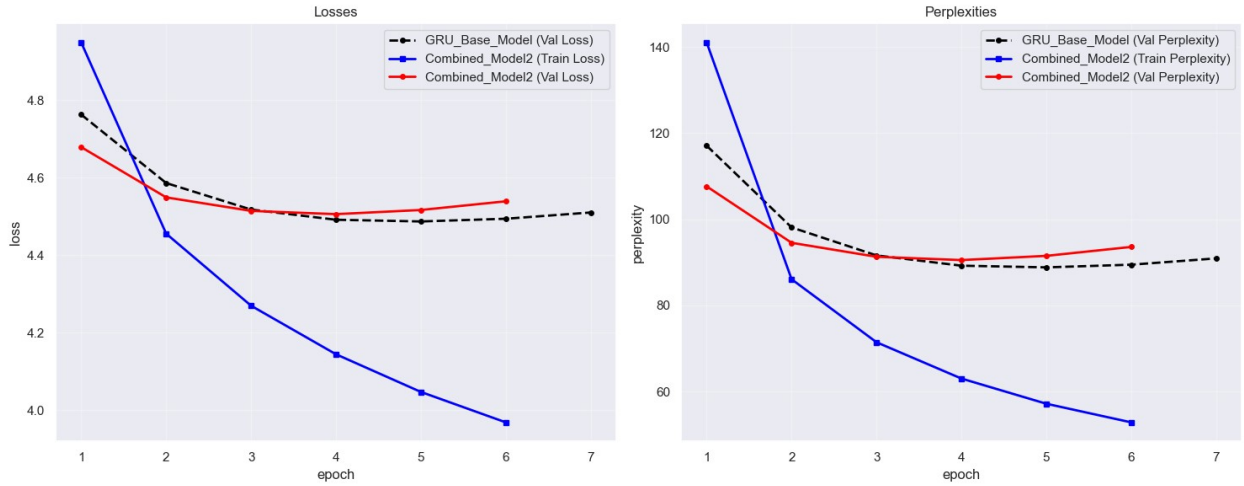
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```



```
Loss = 4.505744769023015
Perplexity = 90.53574716584137

Сравнение с GRU_Base_Model:
Loss Хуже на 0.018857384339357353
Perplexity Хуже на 1.6912707912636193

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	True
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398278	True
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	True
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	False

Вариант с GELU

```
class Combined_GELU_LM2(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
```

```
                            batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model2'

combined_gelu_model2 = Combined_GELU_LM2(hidden_dim=512,
vocab_size=len(vocab), num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_gelu_model2.parameters())
# optimizer = torch.optim.AdamW(combined_gelu_model.parameters(),
lr=0.001, weight_decay=0.01)

combined_gelu_losses2, combined_gelu_perplexities2 =
train_model(model=combined_gelu_model2,

criterion=criterion,

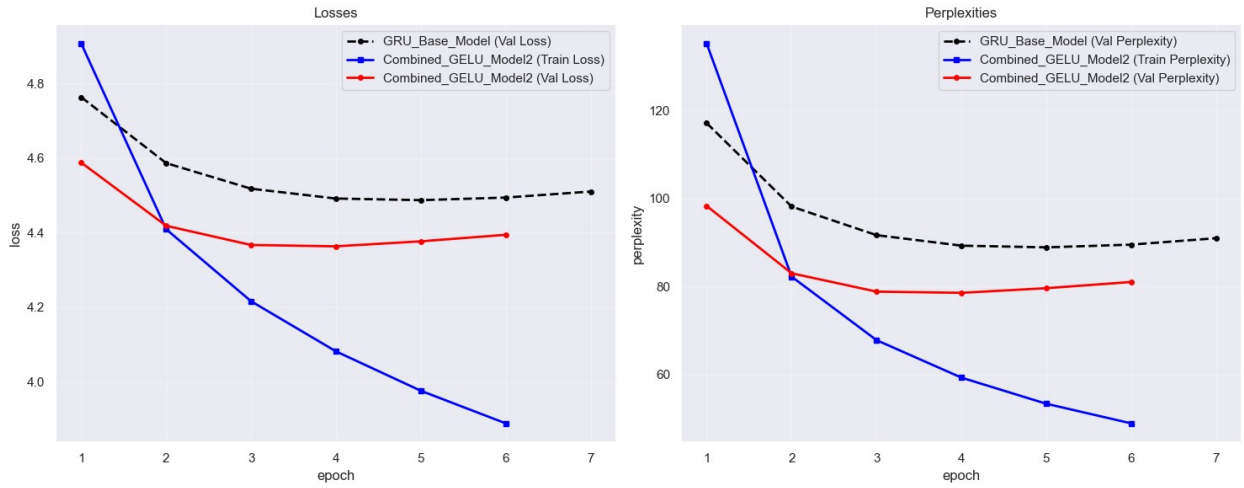
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,
```

```
base_perplexities=base_perplexities)
```




```
Loss = 4.363280537800911
Perplexity = 78.5142814704383

Сравнение с GRU_Base_Model:
Loss Лучше на 0.1236068468827467
Perplexity Лучше на 10.330194904139447

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055041	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921182	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	True
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398277	True
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	True
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	False
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607	-10.330194	True

```
4         -3.055042             True
5          0.815393            False
6         -0.921183             True
7         16.520968            False
8          1.210647            False
9          0.426417            False
10        -0.590572             True
11       -10.398278             True
12       -12.749681             True
13         1.691271            False
14       -10.330195             True
```

Использование другого оптимизатора: AdamW с lr=3e-4 и weight_decay=0.01

```
class Combined_LM3(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)
```

```
model_name = 'Combined_Model3'

combined_model3 = Combined_LM3(hidden_dim=512, vocab_size=len(vocab),
num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(combined_model.parameters())
optimizer = torch.optim.AdamW(combined_model3.parameters(), lr=3e-4,
weight_decay=0.01)

combined_losses3, combined_perplexities3 =
train_model(model=combined_model3,

criterion=criterion,

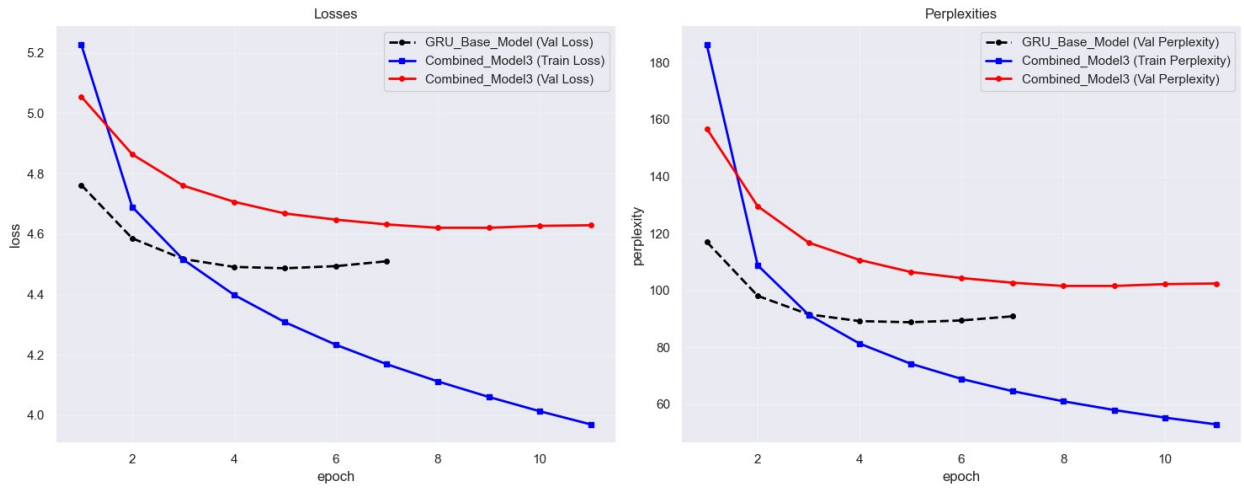
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.620879008219792
Perplexity = 101.58328543949706

Сравнение с GRU_Base_Model:
Loss Хуже на 0.1339916235361347
Perplexity Хуже на 12.738809064919309

display(df_stats)
```

```
                        Model      Loss  Perplexity  Diff_Loss  \
0              GRU_Base_Model  4.486887   88.844476   0.000000
1                  LSTM_Model  4.446406   85.319742  -0.040482
2       MultiLayer_GRU_Model  4.458648   86.370640  -0.028240
3          Stacked_GRU_Model  4.451247   85.733796  -0.035640
4      LargerHiddenDim_Model  4.451896   85.789435  -0.034992
5   LargerLinearLayers_Model  4.496023   89.659869   0.009136
6              GELU_GRU_Model  4.476465   87.923294  -0.010423
7        LargerDropout_Model  4.657435  105.365444   0.170547
8            BatchNorm_Model  4.500422   90.055123   0.013535
9            LayerNorm_Model  4.491675   89.270893   0.004788
10  LargerEmbeddingDim_Model  4.480218   88.253904  -0.006669
11             Combined_Model  4.362413   78.446199  -0.124474
12       Combined_GELU_Model  4.331980   76.094795  -0.154908
13           Combined_Model2  4.505745   90.535747   0.018857
14      Combined_GELU_Model2  4.363281   78.514281  -0.123607
15           Combined_Model3  4.620879  101.583285   0.133992

    Diff_Perplexity  Better_than_base
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
11       -10.398278              True
12       -12.749681              True
13         1.691271             False
14       -10.330195              True
15        12.738809             False
```

Вариант с GELU

```
class Combined_GELU_LM3(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
```

```
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model3'

combined_gelu_model3 = Combined_GELU_LM3(hidden_dim=512,
vocab_size=len(vocab), num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
# optimizer = torch.optim.Adam(combined_gelu_model.parameters())
optimizer = torch.optim.AdamW(combined_gelu_model3.parameters(),
lr=3e-4, weight_decay=0.01)

combined_gelu_losses3, combined_gelu_perplexities3 =
train_model(model=combined_gelu_model3,

criterion=criterion,

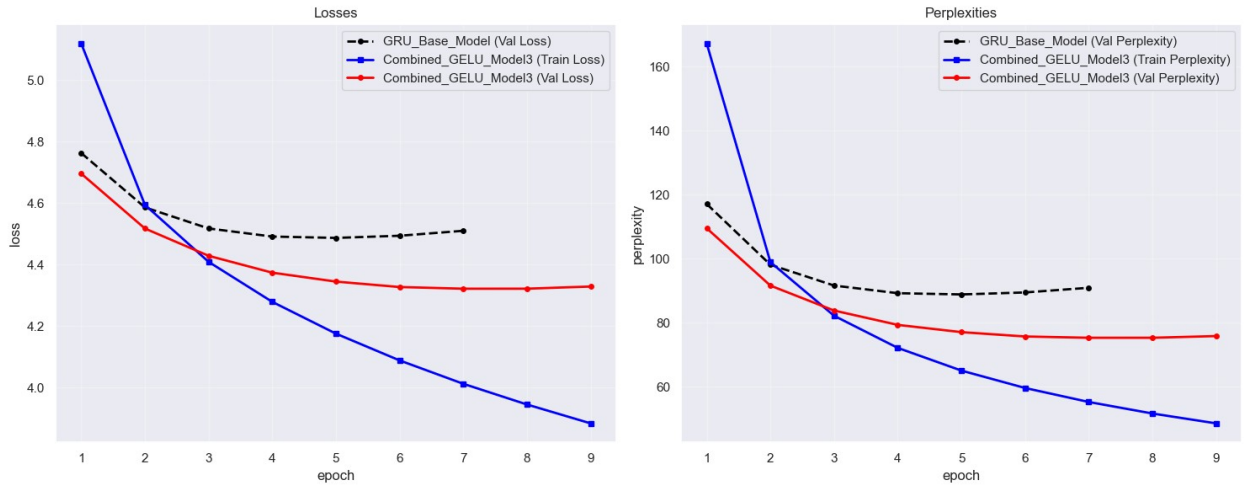
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.321654561238411
Perplexity = 75.31313545592405

Сравнение с GRU_Base_Model:
Loss Лучше на 0.1652328234452467
Perplexity Лучше на 13.531340918653697

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.920782	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.521368	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	True
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398277	True
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	True
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	False
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607	-10.330195	True
15	Combined_Model3	4.620879	101.583285	0.133992	12.738809	False
16	Combined_GELU_Model3	4.321655	75.313135	-0.165233	-13.531341	True

```
6          -0.921183              True
7          16.520968             False
8           1.210647             False
9           0.426417             False
10         -0.590572              True
11        -10.398278              True
12        -12.749681              True
13          1.691271             False
14        -10.330195              True
15         12.738809             False
16        -13.531341              True
```

Уменьшен hidden_dim с 512 до 256

```
class Combined_LM4(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)
```

```
model_name = 'Combined_Model4'

combined_model4 = Combined_LM4(hidden_dim=256, vocab_size=len(vocab),
num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_model4.parameters())
# optimizer = torch.optim.AdamW(combined_model.parameters(), lr=0.001,
weight_decay=0.01)

combined_losses4, combined_perplexities4 =
train_model(model=combined_model4,

criterion=criterion,

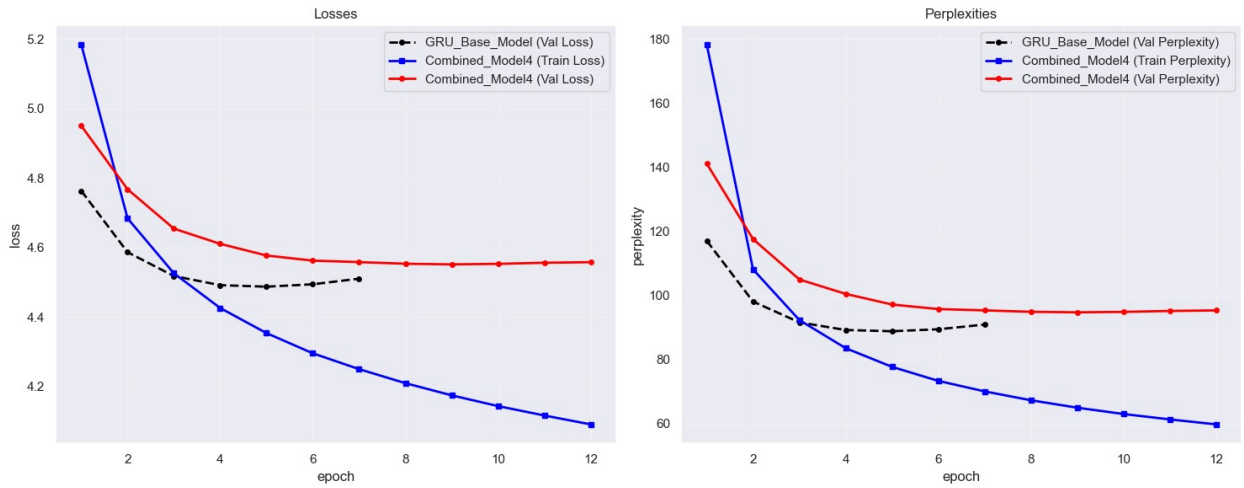
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.551148909788865
Perplexity = 94.74119489619856

Сравнение с GRU_Base_Model:
Loss Хуже на 0.06426152510520744
Perplexity Хуже на 5.896718521620812

display(df_stats)
```

```
                        Model      Loss  Perplexity  Diff_Loss  \
0             GRU_Base_Model  4.486887   88.844476   0.000000
1                 LSTM_Model  4.446406   85.319742  -0.040482
2       MultiLayer_GRU_Model  4.458648   86.370640  -0.028240
3          Stacked_GRU_Model  4.451247   85.733796  -0.035640
4      LargerHiddenDim_Model  4.451896   85.789435  -0.034992
5   LargerLinearLayers_Model  4.496023   89.659869   0.009136
6             GELU_GRU_Model  4.476465   87.923294  -0.010423
7        LargerDropout_Model  4.657435  105.365444   0.170547
8            BatchNorm_Model  4.500422   90.055123   0.013535
9            LayerNorm_Model  4.491675   89.270893   0.004788
10  LargerEmbeddingDim_Model  4.480218   88.253904  -0.006669
11            Combined_Model  4.362413   78.446199  -0.124474
12       Combined_GELU_Model  4.331980   76.094795  -0.154908
13           Combined_Model2  4.505745   90.535747   0.018857
14      Combined_GELU_Model2  4.363281   78.514281  -0.123607
15           Combined_Model3  4.620879  101.583285   0.133992
16      Combined_GELU_Model3  4.321655   75.313135  -0.165233
17           Combined_Model4  4.551149   94.741195   0.064262

    Diff_Perplexity  Better_than_base
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
11       -10.398278              True
12       -12.749681              True
13         1.691271             False
14       -10.330195              True
15        12.738809             False
16       -13.531341              True
17         5.896719             False
```

Вариант с GELU

```
class Combined_GELU_LM4(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
```

```
                            batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model4'

combined_gelu_model4 = Combined_GELU_LM4(hidden_dim=256,
vocab_size=len(vocab), num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_gelu_model4.parameters())
# optimizer = torch.optim.AdamW(combined_gelu_model.parameters(),
lr=0.001, weight_decay=0.01)

combined_gelu_losses4, combined_gelu_perplexities4 =
train_model(model=combined_gelu_model4,

criterion=criterion,

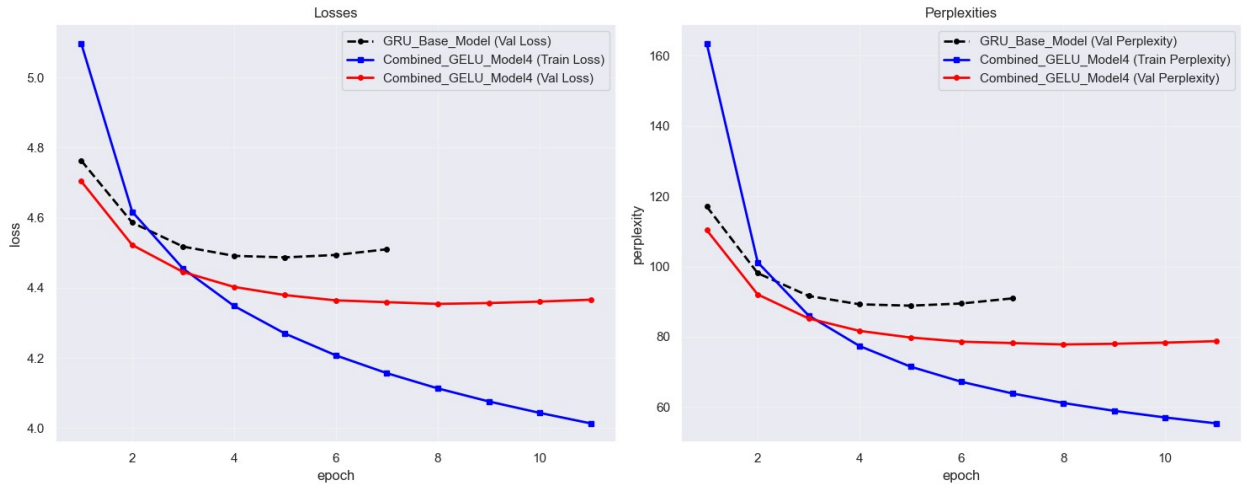
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,
```

```
base_perplexities=base_perplexities)
```




```
Loss = 4.354275966301943
Perplexity = 77.81046753625886

Сравнение с GRU_Base_Model:
Loss Лучше на 0.13261141838171486
Perplexity Лучше на 11.034008838318897

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473836	
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055041	
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921182	
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398277	
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607	-10.330195	
15	Combined_Model3	4.620879	101.583285	0.133992	12.738809	
16	Combined_GELU_Model3	4.321655	75.313135	-0.165233	-13.531341	
17	Combined_Model4	4.551149	94.741195	0.064262	5.896719	
18	Combined_GELU_Model4	4.354276	77.810468	-0.132611	-11.034008	

```
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
11       -10.398278              True
12       -12.749681              True
13         1.691271             False
14       -10.330195              True
15        12.738809             False
16       -13.531341              True
17         5.896719             False
18       -11.034009              True
```

Убрана нелинейность перед последним линейным слоем и увеличенный дропаут (dropout_rate=0.3)

```
class Combined_LM5(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm
```

```
        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(output)

        return projection

set_seed(SEED)

model_name = 'Combined_Model5'

combined_model5 = Combined_LM5(hidden_dim=512, vocab_size=len(vocab),
num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_model5.parameters())
# optimizer = torch.optim.AdamW(combined_model.parameters(), lr=0.001,
weight_decay=0.01)

combined_losses5, combined_perplexities5 =
train_model(model=combined_model5,

criterion=criterion,

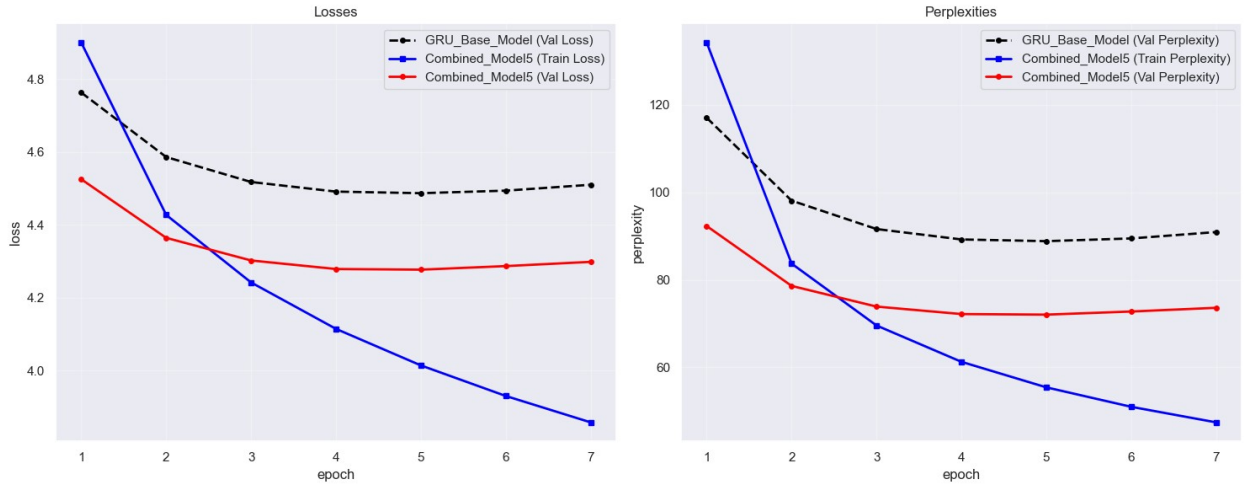
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```



```
Loss = 4.27720355681884
Perplexity = 72.03870592188166

Сравнение с GRU_Base_Model:
Loss Лучше на 0.20968382786481765
Perplexity Лучше на 16.805770452696095

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	True
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398278	True
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	True
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	False
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607	-10.330195	True
15	Combined_Model3	4.620879	101.583285	0.133992	12.738809	False
16	Combined_GELU_Model3	4.321655	75.313135	-0.165233	-13.531341	True
17	Combined_Model4	4.551149	94.741195	0.064262	5.896719	False
18	Combined_GELU_Model4	4.354276	77.810468	-0.132611		
19	Combined_Model5	4.277204	72.038706	-0.209684		

```
18       -11.034009              True
19       -16.805770              True
```

Вариант с GELU

```
class Combined_GELU_LM5(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 2, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(output)

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model5'

combined_gelu_model5 = Combined_GELU_LM5(hidden_dim=512,
vocab_size=len(vocab), num_layers=2, dropout_rate=0.3).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_gelu_model5.parameters())
# optimizer = torch.optim.AdamW(combined_gelu_model.parameters(),
lr=0.001, weight_decay=0.01)
```

```
combined_gelu_losses5, combined_gelu_perplexities5 =
train_model(model=combined_gelu_model5,

criterion=criterion,

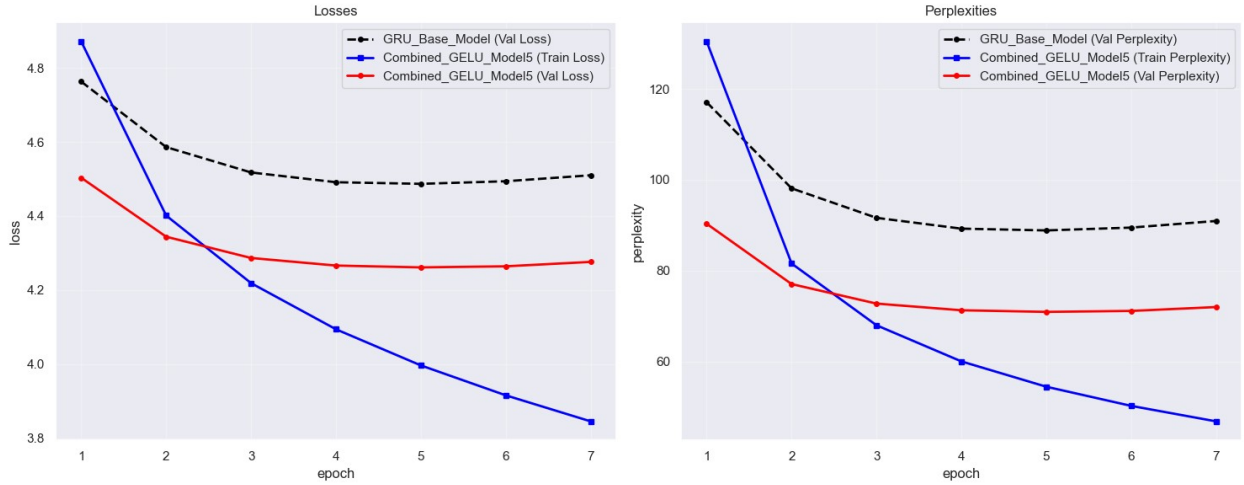
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.261339210546934
Perplexity = 70.90487645761979

Сравнение с GRU_Base_Model:
Loss Лучше на 0.22554817413672357
Perplexity Лучше на 17.93959991695796

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss
0	GRU_Base_Model	4.486887	88.844476	0.000000
1	LSTM_Model	4.446406	85.319742	-0.040482
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136
6	GELU_GRU_Model	4.476465	87.923294	-0.010423
7	LargerDropout_Model	4.657435	105.365444	0.170547
8	BatchNorm_Model	4.500422	90.055123	0.013535

```
9            LayerNorm_Model  4.491675   89.270893   0.004788
10  LargerEmbeddingDim_Model  4.480218   88.253904  -0.006669
11           Combined_Model  4.362413   78.446199  -0.124474
12       Combined_GELU_Model  4.331980   76.094795  -0.154908
13           Combined_Model2  4.505745   90.535747   0.018857
14       Combined_GELU_Model2  4.363281   78.514281  -0.123607
15           Combined_Model3  4.620879  101.583285   0.133992
16       Combined_GELU_Model3  4.321655   75.313135  -0.165233
17           Combined_Model4  4.551149   94.741195   0.064262
18       Combined_GELU_Model4  4.354276   77.810468  -0.132611
19           Combined_Model5  4.277204   72.038706  -0.209684
20       Combined_GELU_Model5  4.261339   70.904876  -0.225548

    Diff_Perplexity  Better_than_base
0          0.000000             False
1         -3.524734              True
2         -2.473837              True
3         -3.110680              True
4         -3.055042              True
5          0.815393             False
6         -0.921183              True
7         16.520968             False
8          1.210647             False
9          0.426417             False
10        -0.590572              True
11       -10.398278              True
12       -12.749681              True
13         1.691271             False
14       -10.330195              True
15        12.738809             False
16       -13.531341              True
17         5.896719             False
18       -11.034009              True
19       -16.805770              True
20       -17.939600              True
```

Убрана нелинейность перед последним линейным слоем, дропаут не увеличен (dropout_rate=0.1)

```
class Combined_LM6(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 1, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)
```

```
        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.Tanh()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(output)

        return projection

set_seed(SEED)

model_name = 'Combined_Model6'

combined_model6 = Combined_LM6(hidden_dim=512, vocab_size=len(vocab),
num_layers=2).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_model6.parameters())
# optimizer = torch.optim.AdamW(combined_model.parameters(), lr=0.001,
weight_decay=0.01)

combined_losses6, combined_perplexities6 =
train_model(model=combined_model6,

criterion=criterion,

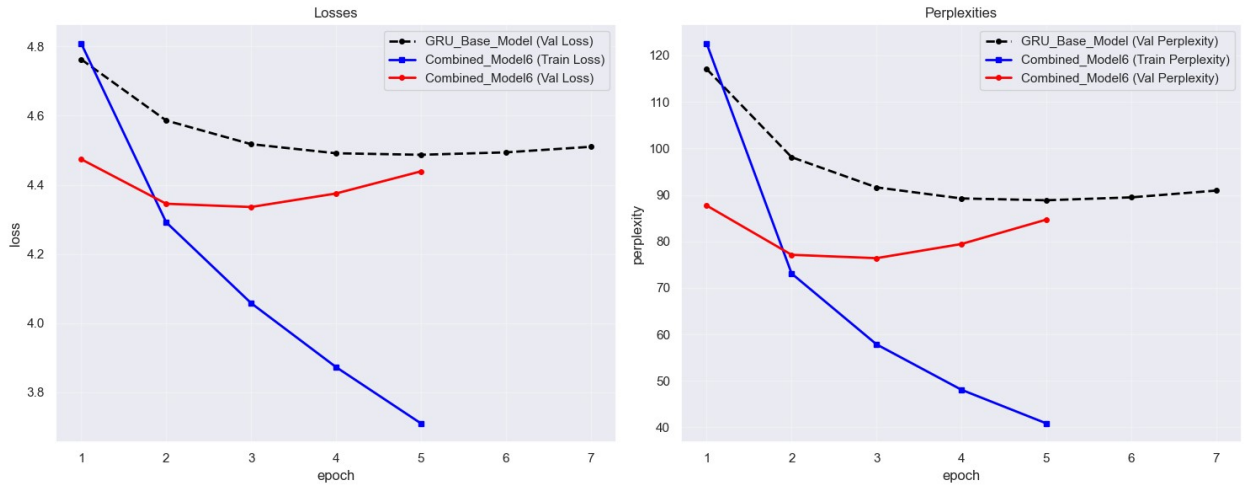
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```




```
Loss = 4.335825476890955
Perplexity = 76.38798939106628

Сравнение с GRU_Base_Model:
Loss Лучше на 0.15106190779270268
Perplexity Лучше на 12.456486983511468

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482		
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240		
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640		
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992		
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136		
6	GELU_GRU_Model	4.476465	87.923294	-0.010423		
7	LargerDropout_Model	4.657435	105.365444	0.170547		
8	BatchNorm_Model	4.500422	90.055123	0.013535		
9	LayerNorm_Model	4.491675	89.270893	0.004788		
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669		
11	Combined_Model	4.362413	78.446199	-0.124474		
12	Combined_GELU_Model	4.331980	76.094795	-0.154908		
13	Combined_Model2	4.505745	90.535747	0.018857		
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607		
15	Combined_Model3	4.620879	101.583285	0.133992		
16	Combined_GELU_Model3	4.321655	75.313135	-0.165233		
17	Combined_Model4	4.551149	94.741195	0.064262		
18	Combined_GELU_Model4	4.354276	77.810468	-0.132611		
19	Combined_Model5	4.277204	72.038706	-0.209684		
20	Combined_GELU_Model5	4.261339	70.904876	-0.225548		
21	Combined_Model6	4.335825	76.387989	-0.151062		

```
1          -3.524734              True
2          -2.473837              True
3          -3.110680              True
4          -3.055042              True
5           0.815393             False
6          -0.921183              True
7          16.520968             False
8           1.210647             False
9           0.426417             False
10         -0.590572              True
11        -10.398278              True
12        -12.749681              True
13          1.691271             False
14        -10.330195              True
15         12.738809             False
16        -13.531341              True
17          5.896719             False
18        -11.034009              True
19        -16.805770              True
20        -17.939600              True
21        -12.456487              True
```

Вариант с GELU

```
class Combined_GELU_LM6(nn.Module):
    def __init__(self, hidden_dim: int, vocab_size: int, num_layers:
int = 1, dropout_rate: float = 0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.rnn = nn.LSTM(hidden_dim, hidden_dim,
num_layers=num_layers,
                           batch_first=True, dropout=dropout_rate if
num_layers > 1 else 0)

        # self.batch_norm = nn.BatchNorm1d(hidden_dim)    # BatchNorm
        self.layer_norm = nn.LayerNorm(hidden_dim)      # LayerNorm

        self.linear = nn.Linear(hidden_dim, hidden_dim)
        self.projection = nn.Linear(hidden_dim, vocab_size)

        self.non_lin = nn.GELU()
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, input_batch) -> torch.Tensor:
        embeddings = self.embedding(input_batch)
        output, _ = self.rnn(embeddings)

        # output = output.transpose(1, 2)    # BatchNorm
        # output = self.batch_norm(output)    # BatchNorm
```

```
        # output = output.transpose(1, 2)    # BatchNorm

        output = self.layer_norm(output)    # LayerNorm

        output = self.dropout(self.linear(self.non_lin(output)))
        projection = self.projection(self.non_lin(output))

        return projection

set_seed(SEED)

model_name = 'Combined_GELU_Model6'

combined_gelu_model6 = Combined_GELU_LM6(hidden_dim=512,
vocab_size=len(vocab), num_layers=2).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=word2ind['<pad>'])
optimizer = torch.optim.Adam(combined_gelu_model6.parameters())
# optimizer = torch.optim.AdamW(combined_gelu_model.parameters(),
lr=0.001, weight_decay=0.01)

combined_gelu_losses6, combined_gelu_perplexities6 =
train_model(model=combined_gelu_model6,

criterion=criterion,

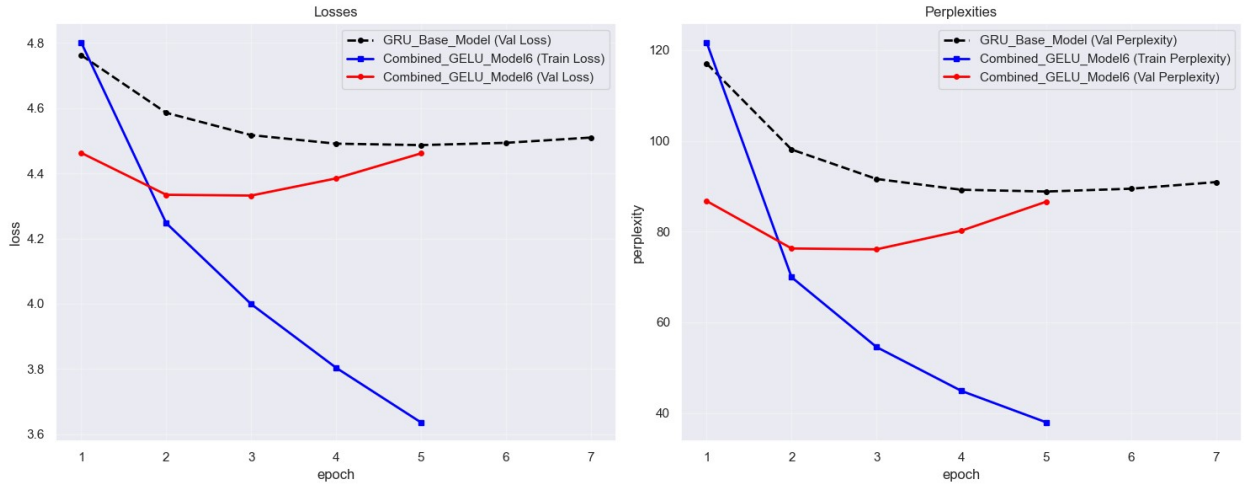
optimizer=optimizer,

model_name=model_name,
                                                    plot=True,

num_epoch=NUM_EPOCH,

base_losses=base_losses,

base_perplexities=base_perplexities)
```



```
Loss = 4.331979864682907
Perplexity = 76.09479492395792

Сравнение с GRU_Base_Model:
Loss Лучше на 0.1549075200007506
Perplexity Лучше на 12.749681450619832

display(df_stats)
```

	Model	Loss	Perplexity	Diff_Loss	Diff_Perplexity	Better_than_base
0	GRU_Base_Model	4.486887	88.844476	0.000000	0.000000	False
1	LSTM_Model	4.446406	85.319742	-0.040482	-3.524734	True
2	MultiLayer_GRU_Model	4.458648	86.370640	-0.028240	-2.473837	True
3	Stacked_GRU_Model	4.451247	85.733796	-0.035640	-3.110680	True
4	LargerHiddenDim_Model	4.451896	85.789435	-0.034992	-3.055042	True
5	LargerLinearLayers_Model	4.496023	89.659869	0.009136	0.815393	False
6	GELU_GRU_Model	4.476465	87.923294	-0.010423	-0.921183	True
7	LargerDropout_Model	4.657435	105.365444	0.170547	16.520968	False
8	BatchNorm_Model	4.500422	90.055123	0.013535	1.210647	False
9	LayerNorm_Model	4.491675	89.270893	0.004788	0.426417	False
10	LargerEmbeddingDim_Model	4.480218	88.253904	-0.006669	-0.590572	True
11	Combined_Model	4.362413	78.446199	-0.124474	-10.398278	True
12	Combined_GELU_Model	4.331980	76.094795	-0.154908	-12.749681	True
13	Combined_Model2	4.505745	90.535747	0.018857	1.691271	False
14	Combined_GELU_Model2	4.363281	78.514281	-0.123607	-10.330195	True
15	Combined_Model3	4.620879	101.583285	0.133992		
16	Combined_GELU_Model3	4.321655	75.313135	-0.165233		
17	Combined_Model4	4.551149	94.741195	0.064262		
18	Combined_GELU_Model4	4.354276	77.810468	-0.132611		
19	Combined_Model5	4.277204	72.038706	-0.209684		
20	Combined_GELU_Model5	4.261339	70.904876	-0.225548		
21	Combined_Model6	4.335825	76.387989	-0.151062		
22	Combined_GELU_Model6	4.331980	76.094795	-0.154908		

```
15        12.738809             False
16       -13.531341              True
17         5.896719             False
18       -11.034009              True
19       -16.805770              True
20       -17.939600              True
21       -12.456487              True
22       -12.749681              True
```

Отчет (2 балла)

Опишите проведенные эксперименты. Сравните перплексии полученных моделей. Предложите идеи по улучшению качества моделей.

Словарь vocab_size уменьшен с 40000 до 20000 слов в целях уменьшения переобучения моделей.

Было реализовано:

- Базовая модель с одним GRU слоем. Все последующие реализации сравнивались с метриками данныой модели.
- Была опробована замена GRU слоя на LSTM слой. Модель показала значительно лучшие результаты. Модель борется с затухающими градиентами и лучше запоминает длинные зависимости.
- Использования двух GRU слоев. Незначительное улучшение метрики Perplexity. Результат, возможно, будет лучше в связке с регуляризациями.
- Увеличение скрытого состояния с 256 до 512. Заметное улучшение относительно базовой модели. Модель смогла выучить более сложные зависимости.
- Два независимых GRU-блока с dropout между ними. Небольшое улучшение относительно базовой модели.
- Добавление дополнительного линейного слоя и небольшим Dropout. Значение метрик хуже, чем в базовой модели.
- Замена функции активации Tanh на GELU. Значение метрик лучше, чем в базовой модели.
- Увеличение Dropout с 0.1 до 0.3. Значение метрик хуже, чем в базовой модели.
- Добавлена Batch Normalization (нормализация активации по батчу). Значение метрик хуже, чем в базовой модели.
- Добавлена Layer Normalization (нормализация активации по каждому элементу батча). Значение метрик лучше, чем в базовой модели. Для RNN моделей предпочтительнее.
- Увеличение размерности эмбеддингов с 256 до 512. Небольшое улучшение относительно базовой модели.

Реализация финальных моделей Combined_Model и Combined_GELU_Model, которые включают лучшие практики из числа проведенных экспериментов:

- Замена GRU слоя на LSTM слой
- Использования двух слоев RNN (два слоя LSTM)

- Использование размера скрытого слоя hidden_dim = 512
- Добавлена Layer Normalization

Результаты получились следующие:
Combined_GELU_Model - перплексия на валидации 76.09, на тесте 76.12,
Combined_Model - перплексия на валидации 78.45, на тесте 78.89.

Далее были продолжены эксперименты с Combined-моделями: изменения дропаута, подбор оптимизатора, скрытых слоев, устранение нелинейности перед последним линейным слоем в модели.

Лучшей реализацией оказалась модель Combined_GELU_Model5 - перплексия на валидации 70.90, на тесте 70.97. В ней dropout = 0.3, GELU, Adam с lr=0.001 и убрана нелинейность перед последним линейным слоем в модели.

Предложения по улучшению:

- Подобрать оптимальный оптимизатор
- Подобрать оптимальные гиперпараметры

График сравнения всех моделей

```
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 7))

# Словарь со всеми моделями и полученными метриками
histories = {
    'GRU_Base_Model': (base_losses, base_perplexities),
    'LSTM_Model': (lstm_losses, lstm_perplexities),
    'MultiLayer_GRU_Model': (multilayer_losses,
multilayer_perplexities),
    'Stacked_GRU_Model': (stacked_gru_losses,
stacked_gru_perplexities),
    'LargerHiddenDim_Model': (larger_hidden_losses,
larger_hidden_perplexities),
    'LargerLinearLayers_Model': (larger_linear_losses,
larger_linear_perplexities),
    'GELU_GRU_Model': (gelu_losses, gelu_perplexities),
    'LargerDropout_Model': (larger_dropout_losses,
larger_dropout_perplexities),
    'BatchNorm_Model': (batchnorm_losses, batchnorm_perplexities),
    'LayerNorm_Model': (layernorm_losses, layernorm_perplexities),
    'LargerEmbeddingDim_Model': (larger_emb_losses,
larger_emb_perplexities),
    'Combined_Model': (combined_losses, combined_perplexities),
    'Combined_GELU_Model': (combined_gelu_losses,
combined_gelu_perplexities),
    'Combined_Model5': (combined_losses5, combined_perplexities5),
    'Combined_GELU_Model5': (combined_gelu_losses5,
combined_gelu_perplexities5)
```

```
}

colors = plt.cm.tab20.colors

for i, (model_name, (losses, perplexities)) in
enumerate(histories.items()):
    color = colors[i % len(colors)]

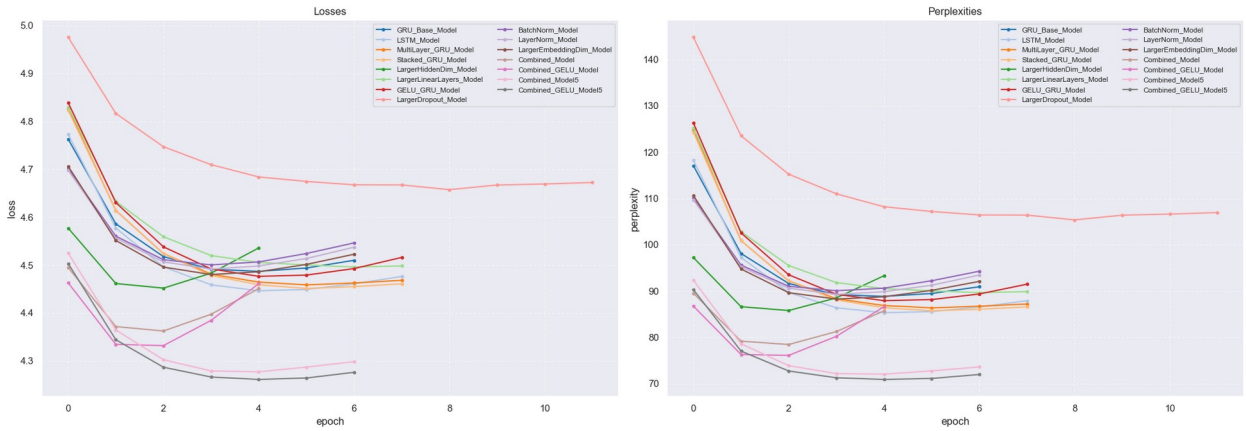
    axes[0].plot(losses, color=color, marker='o', markersize=3,
linewidth=1.5, label=model_name)
    axes[1].plot(perplexities, color=color, marker='o', markersize=3,
linewidth=1.5, label=model_name)

axes[0].set_title('Losses')
axes[0].set_xlabel('epoch')
axes[0].set_ylabel('loss')
axes[0].grid(True, linestyle='--', alpha=0.6)
axes[0].legend(loc='upper right', fontsize='x-small', ncol=2)

axes[1].set_title('Perplexities')
axes[1].set_xlabel('epoch')
axes[1].set_ylabel('perplexity')
axes[1].grid(True, linestyle='--', alpha=0.6)
axes[1].legend(loc='upper right', fontsize='x-small', ncol=2)

plt.tight_layout()

plt.show();
```



Проверка на тесте для combined_model

```
test_loss_combined_model, test_perplexity_combined_model =
evaluate(combined_model, criterion, test_dataloader)

print(f'Loss на тесте: {test_loss_combined_model}')
print(f'Perplexity на тесте: {test_perplexity_combined_model}')
```

```
Loss на тесте: 4.36809563331115
Perplexity на тесте: 78.89324687975966
```

Проверка на тесте для combined_gelu_model

```
test_loss_combined_gelu_model, test_perplexity_combined_gelu_model =
evaluate(combined_gelu_model, criterion, test_dataloader)

print(f'Loss на тесте: {test_loss_combined_gelu_model}')
print(f'Perplexity на тесте: {test_perplexity_combined_gelu_model}')
```

```
Loss на тесте: 4.3323621444213085
Perplexity на тесте: 76.12388998312665
```

Проверка на тесте для combined_model5

```
test_loss_combined_model5, test_perplexity_combined_model5 =
evaluate(combined_model5, criterion, test_dataloader)

print(f'Loss на тесте: {test_loss_combined_model5}')
print(f'Perplexity на тесте: {test_perplexity_combined_model5}')
```

```
Loss на тесте: 4.27549269871834
Perplexity на тесте: 71.9155632881288
```

Проверка на тесте для combined_gelu_model5

```
test_loss_combined_gelu_model5, test_perplexity_combined_gelu_model5 =
evaluate(combined_gelu_model5, criterion, test_dataloader)

print(f'Loss на тесте: {test_loss_combined_gelu_model5}')
print(f'Perplexity на тесте: {test_perplexity_combined_gelu_model5}')
```

```
Loss на тесте: 4.2622070129101095
Perplexity на тесте: 70.96643458325063
```

Попробуем сгенерировать продолжение предложений

```
def generate_sequence(model, starting_seq: str, max_seq_len: int = 50)
-> str:
    device = 'cpu'
    model = model.to(device)

    # Разбиваем начальную строку на слова
    starting_words = word_tokenize(starting_seq.lower())

    input_ids = [word2ind['<bos>']] + [
        word2ind.get(word, word2ind['<unk>']) for word in
starting_words]
```

```
    input_ids = torch.LongTensor(input_ids).to(device)

    model.eval()
    with torch.no_grad():
        for i in range(max_seq_len):
            output = model(input_ids.unsqueeze(0))
            next_word_distribution = output[0, -1, :]
            next_word = next_word_distribution.argmax()
            input_ids = torch.cat([input_ids, next_word.view(1)])

            if next_word.item() == word2ind['<eos>']:
                break

    words = ' '.join([ind2word[idx.item()] for idx in input_ids[1:]
                      if ind2word[idx.item()] not in ['<pad>',
'<unk>', '<eos>']])

    return words
```

combined_model

```
generate_sequence(combined_model, starting_seq='The filmmakers tried
hard, but ')

"the filmmakers tried hard , but the story is so bad that it 's not
funny ."

generate_sequence(combined_model, starting_seq='never seen so many
mistakes')

'never seen so many mistakes in this movie .'

generate_sequence(combined_model, starting_seq='this film')

'this film is a very good movie .'
```

combined_gelu_model

```
generate_sequence(combined_gelu_model, starting_seq='The filmmakers
tried hard, but ')

"the filmmakers tried hard , but they were n't even remotely funny ."

generate_sequence(combined_gelu_model, starting_seq='never seen so
many mistakes')

'never seen so many mistakes in this movie .'

generate_sequence(combined_gelu_model, starting_seq='this film')

'this film is a very good movie .'
```

combined_model5

```
generate_sequence(combined_model5, starting_seq='The filmmakers tried hard, but ')
```

```
"the filmmakers tried hard , but the story was n't even remotely funny ."
```

```
generate_sequence(combined_model5, starting_seq='never seen so many mistakes')
```

```
"never seen so many mistakes , but it 's not even a good movie ."
```

```
generate_sequence(combined_model5, starting_seq='this film')
```

```
'this film is a very good film .'
```

combined_gelu_model5

```
generate_sequence(combined_gelu_model5, starting_seq='The filmmakers tried hard, but ')
```

```
"the filmmakers tried hard , but they were n't funny ."
```

```
generate_sequence(combined_gelu_model5, starting_seq='never seen so many mistakes')
```

```
'never seen so many mistakes .'
```

```
generate_sequence(combined_gelu_model5, starting_seq='this film')
```

```
'this film is a great movie .'
```